ALUMNICONNECT

Relational Design Specification

Keys, functional dependencies, normalization and MySQL implementation

DESIGN PURPOSE

Translate the Alumni–Student Mentorship Platform blueprint into a logical relational design that can be implemented, tested and defended in a DBMS project review.

PROJECT

AlumniConnect - Alumni and Student Mentorship Platform

DOCUMENT VERSION 1.2

DATE 02 November 2026

Prepared as the relational-design baseline, reconciled with the latest AlumniConnect PRD, Conceptual ER Model, Business Rules Catalogue, Application Architecture and DBMS course materials.

Document control and reading guide

Item	Value
Document	Relational Design Specification
Product	AlumniConnect
Version	1.2 - reconciled with the full design pack
Primary DBMS	MySQL 8.x with InnoDB
Source baseline	Project Blueprint, PRD, Conceptual ER Model, Business Rules Catalogue, Application Architecture and DBMS Unit 2/3 materials
Academic alignment	DBMS Unit 3: Functional Dependencies and Normalization; Unit 2: Triggers and Stored Procedures
Status	Design baseline for team review; source-derived rules and recommended implementation decisions are distinguished

How to use this document

Use Sections 3-7 to understand the relational model and the reasoning behind its keys and dependencies. Use Sections 8-10 when writing MySQL DDL, stored procedures, triggers and test cases. The SQL is a consistent implementation template, but should be executed only after the team freezes the final naming and status vocabulary.

Contents

- 1. Executive design summary
- 2. Scope, assumptions and design decisions
- 3. Relational modelling conventions
- 4. Proposed relation catalogue (24 conceptual entities plus one recommended assessment relation)
- 5. Keys, functional dependencies, Armstrong axioms and minimal cover
- 6. Candidate-key derivation and attribute closure
- 7. Normalization walkthrough: 1NF to BCNF, with 4NF/5NF context
- 8. Lossless join, dependency preservation and integrity
- 9. MySQL DDL patterns and representative definitions
- 10. Triggers, stored procedures, transaction boundaries and outbox (including ACID / isolation)
- 11. Indexing, views, CTEs and query optimization
- 12. Test matrix, implementation checklist and open decisions
- Appendix A. Glossary and design references

IMPORTANT INTERPRETATION

The blueprint gives the business entities and major relationships. It does not prescribe every column-level dependency or every database constraint. This document therefore labels added candidate keys, column attributes and validation rules as proposed assumptions to be confirmed with the team.

1. Executive design summary

AlumniConnect is modelled as a relational system of record with a normalized MySQL database. It stores account identity, role-specific profiles, skills and domains, mentorship requests, recommendations, allocations, availability, bookings, sessions, feedback, reviews, notifications and audit history.

Design area	Decision
Primary relational store	MySQL 8.x / InnoDB owns authoritative identities and business records.
Identity	One UserAccount represents one login identity. StudentProfile and MentorProfile extend that identity.
Many-to-many data	Use associative relations such as StudentSkill, MentorSkill, StudentDomainInterest and MentorDomainExpertise.
Mentorship lifecycle	Keep MentorshipRequest separate from MentorAllocation. An accepted request can create at most one allocation.
Scheduling	Keep MentorAvailability, Booking and MentorshipSession separate. A booking records a plan; a session records what actually happened.
Operational history	Use NotificationOutbox for reliable event delivery and AuditLog for append-only change history.
Polyglot persistence	Redis holds rebuildable cache / temporary values. Neo4j holds a graph projection for discovery. Neither replaces MySQL as source of truth.
Normalization target	Aim for 3NF, and BCNF where the business dependencies support it, while preserving lossless joins and preferably dependency preservation.

Core relational domains

Domain	Main relations
Identity and organization	Role, UserAccount, StudentProfile, MentorProfile, Department, Company
Skills and expertise	Skill, StudentSkill, MentorSkill, MentorshipDomain, StudentDomainInterest, MentorDomainExpertise
Discovery and allocation	MentorshipRequest, RecommendationRun, RecommendedMentor, MentorAllocation
Scheduling and sessions	MentorAvailability, Booking, MentorshipSession, SessionSkillAssessment
Feedback and operations	SessionFeedback, MentorReview, Notification, NotificationOutbox, AuditLog

WHY THIS DESIGN WORKS

It separates facts that change at different rates and have different meanings. A profile is not a request; a request is not an allocation; a booking is not a completed session; and a notification event is not the same thing as a user inbox notification.

2. Scope, assumptions and design decisions

The following decisions turn the conceptual model and Business Rules Catalogue into a concrete logical schema. [B] marks a blueprint/source-aligned rule; [R] marks an implementation recommendation added where the sources leave a choice open; [T] marks a technical enforcement mechanism. The newest Business Rules Catalogue is used to resolve lifecycle vocabulary, while deviations made for normalization are explicitly explained.

2.1 Source alignment and important reconciliations

Topic	Decision used in this relational specification	Reason / source reconciliation
Allocation lifecycle	Use ACTIVE -> ENDED.	The current Business Rules Catalogue uses ACTIVE and ENDED. Older draft vocabulary such as COMPLETED / TERMINATED is not used in the canonical schema.
Booking eligibility	BOOKING.allocation_id is mandatory and must reference an ACTIVE allocation.	The core workflow and current booking rules require an accepted allocation. The conceptual ER model marks this link as optional only for a possible alternate appointment mode; that optional mode is excluded from this core Release 1 schema.
Allocation participants	Store request_id once; derive student and mentor through MENTORSHIP_REQUEST.	The conceptual entities are traceable through the originating request. Repeating student/mentor IDs in MENTOR_ALLOCATION creates redundant facts and is not needed for normalized storage.
Booking participants and time	Store allocation_id and availability_id; derive participants and slot times through joins.	The Business Rules Catalogue includes a query-oriented sample that repeats student_id, mentor_id, start_at and end_at in BOOKING. This specification omits those duplicates in the normalized base relation; a view can expose them. If snapshots are required, label them as immutable booking-time snapshots and document the controlled denormalization.
Availability vocabulary	Use OPEN, BOOKED, BLOCKED and EXPIRED.	The current Business Rules Catalogue describes a slot as OPEN before reservation and BOOKED after confirmation; BLOCKED and EXPIRED are recommended non-bookable states.
Notification data	Store one outbox event, then fan out into one or more NOTIFICATION rows.	This preserves the conceptual 1-to-many Outbox-to-Notification relationship. Do not write one outbox row per recipient unless the team deliberately changes the event model.
Business event log	Use AUDIT_LOG for actor/action history and NOTIFICATION_OUTBOX for asynchronous delivery.	The business-rule SQL examples mention business_event_log, but the conceptual entity catalogue contains Audit Log and Notification Outbox. This schema keeps those responsibilities separate without adding a duplicate event table.

NORMALIZATION DECISION

A logical model should not copy student, mentor, slot-time or company-description attributes into every child row when those values are already determined through a parent key. Use joins or privacy-safe views for display. Denormalize only after measuring a real query bottleneck and documenting how the duplicated value stays synchronized.

Assumption	Proposed rule
A1. One primary role [R]	Each UserAccount has one primary Role through role_id. A user may have zero or one StudentProfile and zero or one MentorProfile, but Release 1 assigns one primary operating role. If dual-role users are required, replace role_id with a UserRole bridge and define profile-role consistency. [ER model R01-R03]
A2. Targeted request [B]	A MentorshipRequest identifies one student, one target mentor and one mentorship domain/objective. Open requests not addressed to a mentor are outside this schema.
A3. One allocation per request [B]	A request can result in zero or one MentorAllocation. UNIQUE(request_id) enforces this cardinality.

Assumption	Proposed rule
A4. Allocation participant derivation [R]	MentorAllocation stores request_id, lifecycle fields and allocation metadata. Student and mentor are derived from the originating request, avoiding duplicate facts.
A5. Recommendation history [B/R]	Each RecommendationRun belongs to one student. A mentor appears at most once per run and each rank position is unique within that run. Scores and explanations are retained per run.
A6. Availability identity [B/R]	A mentor may publish many slots. Exact duplicate intervals are prohibited by UNIQUE(mentor_user_id, starts_at, ends_at); overlapping OPEN slots require a locked procedure because UNIQUE cannot detect interval overlap.
A7. Booking eligibility [B/R]	Every core Release 1 booking references one ACTIVE allocation and one availability slot owned by that allocation mentor. Booking participants and interval are derived through Allocation/Request and Availability.
A8. Booking history and slot reuse [B/R]	Cancelled bookings remain in history. A generated active_availability_id is non-NULL only for CONFIRMED bookings, allowing one active booking per slot while retaining cancelled rows. Reopening a slot after cancellation is policy-controlled.
A9. Participant overlap [B]	A student and a mentor must each have no other CONFIRMED booking overlapping the requested half-open interval [start, end). A transaction locks participant profile rows before checking.
A10. Session creation [B/R]	A confirmed booking creates at most one SCHEDULED MentorshipSession; the implementation inserts it in the booking transaction. Cancelled bookings need not have a session.
A11. Feedback cardinality [B/R]	SessionFeedback may be submitted by either actual participant after a completed session; enforce at most one row per participant per session. MentorReview is a separate student-authored rating, at most one per completed session.
A12. Controlled vocabularies [R]	Roles, skills and mentorship domains are reference data. Skill and domain labels are canonicalized and unique within the project scope.
A13. Historical snapshots [R]	A request may preserve career_goal_snapshot at submission. This is a deliberate historical snapshot, not a live copy of StudentProfile.career_goal.
A14. History retention [B/R]	Requests, allocations, bookings, sessions, reviews and audit evidence are status-changed or moderated rather than physically deleted in normal operations.
A15. Event fan-out [B/R]	NotificationOutbox stores a durable domain event. One event can create multiple Notification inbox rows, one per recipient; dedupe keys prevent duplicate creation/delivery.
A16. Department key [R]	department_code is treated as unique within the single-institution project. For multi-institution use, replace UNIQUE(department_code) with UNIQUE(institution_id, department_code).

CONFIRM BEFORE BUILDING

Confirm whether accounts can have multiple roles, whether a student can have several active mentors, whether cancelled slots are reusable, and whether feedback is one-way or two-way. These choices affect uniqueness constraints and lifecycle rules.

3. Relational modelling conventions

Notation	Meaning
PK	Primary key: minimal attribute set identifying one row.
FK	Foreign key: attribute(s) referencing a primary or alternate unique key in another relation.
UQ	Unique alternate/candidate key enforced with a UNIQUE constraint.
NN	NOT NULL; a value is mandatory.
NULL	Optional value; only use where the business lifecycle permits absence.
(A, B)	Composite key formed by more than one attribute.
1:N	One parent row may relate to many child rows.
0..1	Optional zero-or-one participation.
JSON	Opaque operational payload; do not use JSON arrays as a substitute for relational bridge tables.

Key terminology

- Superkey: any attribute set that uniquely identifies each legal tuple. A superkey may contain extra attributes.
- Candidate key: a minimal superkey. Removing any attribute causes uniqueness to fail.
- Primary key: the candidate key selected as the main row identifier.
- Alternate key: a candidate key not selected as the primary key; normally enforced using UNIQUE.
- Prime attribute: an attribute that belongs to at least one candidate key. A non-prime attribute belongs to no candidate key.
- Functional dependency X -> Y: when two valid rows agree on X, they must also agree on Y.
- Full functional dependency: Y depends on the whole determinant X; no proper subset of X determines Y.
- Partial dependency: some proper subset of a composite key determines a non-prime attribute.
- Transitive dependency: key -> intermediate non-key -> another non-prime attribute.

Naming and datatype conventions

Convention	Recommendation
Identifiers	BIGINT UNSIGNED AUTO_INCREMENT for surrogate IDs; bridge tables use composite primary keys.
Email	VARCHAR(254), NOT NULL, UNIQUE; normalize case before storage or use a consistent collation.
Timestamps	DATETIME(6) stored consistently in UTC; application converts for display.
Amounts / scores	Use DECIMAL rather than FLOAT for values requiring predictable decimal precision.
Boolean flags	BOOLEAN / TINYINT(1), with explicit defaults and CHECK where supported.
Status	Use an ENUM or a lookup table; status values must be treated as finite lifecycle vocabularies.
Long-form notes	TEXT; split into separate fields only when they have independent meaning or querying needs.
JSON	Use only for flexible snapshots / event payloads, not lists of skills or normalized attributes.

4. Proposed relation catalogue

The 24 conceptual entities from the ER model are mapped to relational tables. SESSION_SKILL_ASSESSMENT is a 25th recommended detail relation so a session can assess multiple skills without storing a repeating list in one cell. The base schema is normalized; derived participant and slot details are exposed through joins or views.

4.1 Identity and organization

ROLE

Defines the platform roles available for authorization.

Specification item	Definition
Attributes	role_id BIGINT; role_code VARCHAR(30); role_name VARCHAR(80); description TEXT; is_active BOOLEAN
Primary key	role_id
Candidate / alternate keys	role_code
Foreign keys	None
Principal functional dependencies	role_id -> role_code, role_name, description, is_active; role_code -> role_id, role_name, description, is_active
Design notes	role_code is a controlled unique code such as STUDENT, MENTOR or ADMIN.

USER_ACCOUNT

Common login and identity record for every platform account.

Specification item	Definition
Attributes	user_id BIGINT; role_id BIGINT; email VARCHAR(254); password_hash VARCHAR(255); display_name VARCHAR(120); account_status VARCHAR(20); created_at DATETIME; updated_at DATETIME; last_login_at DATETIME NULL
Primary key	user_id
Candidate / alternate keys	email
Foreign keys	role_id -> ROLE.role_id
Principal functional dependencies	user_id -> role_id, email, password_hash, display_name, account_status, created_at, updated_at, last_login_at; email -> user_id, role_id, password_hash, display_name, account_status, created_at, updated_at, last_login_at
Design notes	Store only a password hash, never a plaintext password. Email is an alternate candidate key if the system guarantees uniqueness.

STUDENT_PROFILE

Student-specific academic and mentoring information, extending a common account.

Specification item	Definition
Attributes	user_id BIGINT; student_number VARCHAR(50); department_id BIGINT; programme VARCHAR(120); batch_year SMALLINT; expected_graduation_year SMALLINT NULL; career_goal TEXT NULL; bio TEXT NULL
Primary key	user_id

Specification item	Definition
Candidate / alternate keys	student_number
Foreign keys	user_id -> USER_ACCOUNT.user_id; department_id -> DEPARTMENT.department_id
Principal functional dependencies	user_id -> student_number, department_id, programme, batch_year, expected_graduation_year, career_goal, bio; student_number -> user_id, department_id, programme, batch_year, expected_graduation_year, career_goal, bio
Design notes	The student number is proposed as a unique alternate key. Confirm its uniqueness and format with the institution.

MENTOR_PROFILE

Professional and alumni-specific information for a mentor account.

Specification item	Definition
Attributes	user_id BIGINT; alumni_identifier VARCHAR(60) NULL; alumni_department_id BIGINT; current_company_id BIGINT NULL; job_title VARCHAR(120); years_experience DECIMAL(4,1); max_active_mentees SMALLINT; accepting_requests BOOLEAN; professional_summary TEXT NULL
Primary key	user_id
Candidate / alternate keys	alumni_identifier (when non-NULL)
Foreign keys	user_id -> USER_ACCOUNT.user_id; alumni_department_id -> DEPARTMENT.department_id; current_company_id -> COMPANY.company_id
Principal functional dependencies	user_id -> alumni_identifier, alumni_department_id, current_company_id, job_title, years_experience, max_active_mentees, accepting_requests, professional_summary; non-NULL alumni_identifier -> user_id, ...
Design notes	Company is optional because an alumnus may be between roles or self-employed. Alumni identifier is optional and unique only when supplied.

DEPARTMENT

Reference entity for academic departments.

Specification item	Definition
Attributes	department_id BIGINT; department_code VARCHAR(30); department_name VARCHAR(120); institution_name VARCHAR(160)
Primary key	department_id
Candidate / alternate keys	(institution_name, department_code)
Foreign keys	None
Principal functional dependencies	department_id -> department_code, department_name, institution_name; (institution_name, department_code) -> department_id, department_name
Design notes	The natural alternate key is scoped to the institution; department_code alone is not assumed globally unique.

COMPANY

Reference entity for mentor employers.

Specification item	Definition
Attributes	company_id BIGINT; company_name VARCHAR(180); website_url VARCHAR(255) NULL; industry VARCHAR(100) NULL
Primary key	company_id
Candidate / alternate keys	None assumed
Foreign keys	None
Principal functional dependencies	company_id -> company_name, website_url, industry
Design notes	Do not assume company_name is unique; duplicate display names may exist. Use a stable company_id.

4.2 Skills and mentorship domains

SKILL

Controlled vocabulary of technical and professional skills.

Specification item	Definition
Attributes	skill_id BIGINT; skill_name VARCHAR(100); category VARCHAR(80) NULL; description TEXT NULL
Primary key	skill_id
Candidate / alternate keys	skill_name
Foreign keys	None
Principal functional dependencies	skill_id -> skill_name, category, description; skill_name -> skill_id, category, description
Design notes	Normalize skill spelling and aliases before insert. A canonical skill name should appear once.

STUDENT_SKILL

Associative relation connecting students to skills they possess.

Specification item	Definition
Attributes	student_user_id BIGINT; skill_id BIGINT; proficiency_level VARCHAR(20); years_experience DECIMAL(4,1) NULL; added_at DATETIME
Primary key	(student_user_id, skill_id)
Candidate / alternate keys	Composite PK is the candidate key
Foreign keys	student_user_id -> STUDENT_PROFILE.user_id; skill_id -> SKILL.skill_id
Principal functional dependencies	(student_user_id, skill_id) -> proficiency_level, years_experience, added_at
Design notes	Proficiency belongs to the student-skill pair, not to the Skill entity globally.

MENTOR_SKILL

Associative relation connecting mentors to skills they can teach or demonstrate.

Specification item	Definition
Attributes	mentor_user_id BIGINT; skill_id BIGINT; proficiency_level VARCHAR(20); years_experience DECIMAL(4,1) NULL; added_at DATETIME
Primary key	(mentor_user_id, skill_id)
Candidate / alternate keys	Composite PK is the candidate key
Foreign keys	mentor_user_id -> MENTOR_PROFILE.user_id; skill_id -> SKILL.skill_id
Principal functional dependencies	(mentor_user_id, skill_id) -> proficiency_level, years_experience, added_at
Design notes	A mentor may have many skills and one skill may be associated with many mentors.

MENTORSHIP_DOMAIN

Controlled vocabulary for career and mentorship domains.

Specification item	Definition
Attributes	domain_id BIGINT; domain_name VARCHAR(120); description TEXT NULL
Primary key	domain_id
Candidate / alternate keys	domain_name
Foreign keys	None
Principal functional dependencies	domain_id -> domain_name, description; domain_name -> domain_id, description
Design notes	Examples may include Cloud Computing, Machine Learning or Data Analytics. Treat labels as reference data.

STUDENT_DOMAIN_INTEREST

Associative relation recording a student interest in a domain.

Specification item	Definition
Attributes	student_user_id BIGINT; domain_id BIGINT; priority_order SMALLINT; interest_notes TEXT NULL
Primary key	(student_user_id, domain_id)
Candidate / alternate keys	(student_user_id, priority_order), if priority is unique per student
Foreign keys	student_user_id -> STUDENT_PROFILE.user_id; domain_id -> MENTORSHIP_DOMAIN.domain_id
Principal functional dependencies	(student_user_id, domain_id) -> priority_order, interest_notes; (student_user_id, priority_order) -> domain_id, interest_notes (if enforced)
Design notes	A student can express interest in multiple domains. Priority ordering is an additional proposed feature.

MENTOR_DOMAIN_EXPERTISE

Associative relation describing a mentor's expertise in a domain.

Specification item	Definition
Attributes	mentor_user_id BIGINT; domain_id BIGINT; expertise_level VARCHAR(20); years_experience DECIMAL(4,1) NULL; is_primary BOOLEAN
Primary key	(mentor_user_id, domain_id)
Candidate / alternate keys	Composite PK is the candidate key
Foreign keys	mentor_user_id -> MENTOR_PROFILE.user_id; domain_id -> MENTORSHIP_DOMAIN.domain_id
Principal functional dependencies	(mentor_user_id, domain_id) -> expertise_level, years_experience, is_primary
Design notes	The profile can represent multiple specializations. If only one primary domain is permitted, enforce that rule in a controlled procedure or with a suitable unique strategy.

4.3 Discovery, requests and allocation

MENTORSHIP_REQUEST

A student's request to a selected mentor for guidance or mock-interview support.

Specification item	Definition
Attributes	request_id BIGINT; student_user_id BIGINT; mentor_user_id BIGINT; domain_id BIGINT; request_message TEXT; career_goal_snapshot TEXT NULL; request_status VARCHAR(20); submitted_at DATETIME; decision_at DATETIME NULL; decided_by_user_id BIGINT NULL; decision_reason VARCHAR(120) NULL; response_note TEXT NULL; idempotency_key VARCHAR(100) NULL; pending_request_key VARCHAR(100) GENERATED NULL
Primary key	request_id
Candidate / alternate keys	idempotency_key (when non-NULL); pending_request_key is unique only while PENDING
Foreign keys	student_user_id -> STUDENT_PROFILE.user_id; mentor_user_id -> MENTOR_PROFILE.user_id; domain_id -> MENTORSHIP_DOMAIN.domain_id; decided_by_user_id -> USER_ACCOUNT.user_id
Principal functional dependencies	request_id -> student_user_id, mentor_user_id, domain_id, request_message, career_goal_snapshot, request_status, submitted_at, decision_at, decided_by_user_id, decision_reason, response_note, idempotency_key; idempotency_key -> request_id, ...
Design notes	The generated pending_request_key represents the student/mentor/domain combination only while PENDING; it is a partial uniqueness guard, not a candidate key over all request history. Resolved requests remain in history.

RECOMMENDATION_RUN

Records one execution of the mentor recommendation process for a student.

Specification item	Definition
Attributes	run_id BIGINT; student_user_id BIGINT; input_goal TEXT NULL; algorithm_version VARCHAR(40); profile_version VARCHAR(40) NULL; filter_snapshot JSON NULL; executed_at DATETIME
Primary key	run_id
Candidate / alternate keys	None assumed
Foreign keys	student_user_id -> STUDENT_PROFILE.user_id
Principal functional dependencies	run_id -> student_user_id, input_goal, algorithm_version, profile_version, filter_snapshot, executed_at
Design notes	Store a run so recommendation results can be evaluated over time rather than overwritten.

RECOMMENDED_MENTOR

One candidate mentor produced by a recommendation run, with ranking and explanation.

Specification item	Definition
Attributes	recommendation_id BIGINT; run_id BIGINT; mentor_user_id BIGINT; rank_position SMALLINT; score DECIMAL(7,4); eligibility_status VARCHAR(20); explanation_text TEXT; created_at DATETIME
Primary key	recommendation_id
Candidate / alternate keys	(run_id, mentor_user_id); (run_id, rank_position) if rank positions are unique
Foreign keys	run_id -> RECOMMENDATION_RUN.run_id; mentor_user_id -> MENTOR_PROFILE.user_id

Specification item	Definition
Principal functional dependencies	recommendation_id -> run_id, mentor_user_id, rank_position, score, eligibility_status, explanation_text, created_at; (run_id, mentor_user_id) -> rank_position, score, eligibility_status, explanation_text, created_at
Design notes	MySQL must make final eligibility decisions such as capacity; Neo4j can discover relevant relationship paths.

MENTOR_ALLOCATION

The accepted mentorship relationship created from one accepted request.

Specification item	Definition
Attributes	allocation_id BIGINT; request_id BIGINT; allocation_status VARCHAR(20); allocated_at DATETIME; ended_at DATETIME NULL; ended_by_user_id BIGINT NULL; termination_reason TEXT NULL
Primary key	allocation_id
Candidate / alternate keys	request_id
Foreign keys	request_id -> MENTORSHIP_REQUEST.request_id; ended_by_user_id -> USER_ACCOUNT.user_id
Principal functional dependencies	allocation_id -> request_id, allocation_status, allocated_at, ended_at, ended_by_user_id, termination_reason; request_id -> allocation_id, allocation_status, allocated_at, ended_at, ended_by_user_id, termination_reason
Design notes	Store request_id once. Derive student and mentor through MENTORSHIP_REQUEST. Current lifecycle vocabulary is ACTIVE -> ENDED.

4.4 Availability, booking and sessions

MENTOR_AVAILABILITY

A time interval published by a mentor as available for booking.

Specification item	Definition
Attributes	availability_id BIGINT; mentor_user_id BIGINT; starts_at DATETIME; ends_at DATETIME; slot_status VARCHAR(20); created_at DATETIME
Primary key	availability_id
Candidate / alternate keys	(mentor_user_id, starts_at, ends_at)
Foreign keys	mentor_user_id -> MENTOR_PROFILE.user_id
Principal functional dependencies	availability_id -> mentor_user_id, starts_at, ends_at, slot_status, created_at; (mentor_user_id, starts_at, ends_at) -> availability_id, slot_status, created_at
Design notes	Use OPEN, BOOKED, BLOCKED and EXPIRED. Exact duplicate intervals are prevented by UNIQUE; interval overlap must be checked by a locked procedure.

BOOKING

A confirmed or cancelled reservation of a mentor availability slot for an allocation.

Specification item	Definition
Attributes	booking_id BIGINT; allocation_id BIGINT; availability_id BIGINT; booking_status VARCHAR(20); booked_at DATETIME; cancelled_at DATETIME NULL; cancelled_by_user_id BIGINT NULL; cancellation_reason TEXT NULL; idempotency_key VARCHAR(100) NOT NULL; active_availability_id BIGINT GENERATED NULLABLE
Primary key	booking_id
Candidate / alternate keys	idempotency_key; active_availability_id is unique only for CONFIRMED rows, not a candidate key over all history
Foreign keys	allocation_id -> MENTOR_ALLOCATION.allocation_id; availability_id -> MENTOR_AVAILABILITY.availability_id; cancelled_by_user_id -> USER_ACCOUNT.user_id
Principal functional dependencies	booking_id -> allocation_id, availability_id, booking_status, booked_at, cancelled_at, cancelled_by_user_id, cancellation_reason, idempotency_key
Design notes	A core booking requires one ACTIVE allocation and one mentor slot. Participants and times are derived through allocation/request and availability. The required idempotency key makes safe request retries possible. A generated active-slot key prevents double booking while preserving cancelled history.

MENTORSHIP_SESSION

The actual meeting/outcome associated with a confirmed booking.

Specification item	Definition
Attributes	session_id BIGINT; booking_id BIGINT; session_status VARCHAR(20); actual_start_at DATETIME NULL; actual_end_at DATETIME NULL; discussion_summary TEXT NULL; performance_observations TEXT NULL; improvement_areas TEXT NULL; next_steps TEXT NULL; preparation_recommendation TEXT NULL; recorded_by_user_id BIGINT NULL
Primary key	session_id
Candidate / alternate keys	booking_id

Specification item	Definition
Foreign keys	booking_id -> BOOKING.booking_id; recorded_by_user_id -> USER_ACCOUNT.user_id
Principal functional dependencies	session_id -> booking_id, session_status, actual_start_at, actual_end_at, discussion_summary, performance_observations, improvement_areas, next_steps, preparation_recommendation, recorded_by_user_id; booking_id -> session_id, ...
Design notes	A booking is planned; a session captures Scheduled -> Completed / Cancelled / No-show. One session per booking is proposed.

SESSION_SKILL_ASSESSMENT (recommended extension)

Stores one assessed skill per row for a session, avoiding a multi-valued skill list in the session row.

Specification item	Definition
Attributes	session_id BIGINT; skill_id BIGINT; assessment_result VARCHAR(30); assessment_notes TEXT NULL
Primary key	(session_id, skill_id)
Candidate / alternate keys	Composite PK is the candidate key
Foreign keys	session_id -> MENTORSHIP_SESSION.session_id; skill_id -> SKILL.skill_id
Principal functional dependencies	(session_id, skill_id) -> assessment_result, assessment_notes
Design notes	This relation is an additional normalization detail inferred from the blueprint's requirement to record skills assessed.

4.5 Feedback, notifications and audit

SESSION_FEEDBACK

Structured feedback submitted by a real session participant.

Specification item	Definition
Attributes	feedback_id BIGINT; session_id BIGINT; submitted_by_user_id BIGINT; communication_rating TINYINT NULL; helpfulness_rating TINYINT NULL; technical_preparation_rating TINYINT NULL; comments TEXT NULL; submitted_at DATETIME
Primary key	feedback_id
Candidate / alternate keys	(session_id, submitted_by_user_id)
Foreign keys	session_id -> MENTORSHIP_SESSION.session_id; submitted_by_user_id -> USER_ACCOUNT.user_id
Principal functional dependencies	feedback_id -> session_id, submitted_by_user_id, communication_rating, helpfulness_rating, technical_preparation_rating, comments, submitted_at; (session_id, submitted_by_user_id) -> ...
Design notes	A trigger/procedure must confirm the submitter is a session participant and the session has taken place.

MENTOR_REVIEW

A student's rating and review of the mentor after a completed session.

Specification item	Definition
Attributes	review_id BIGINT; session_id BIGINT; rating TINYINT; review_text TEXT NULL; submitted_at DATETIME
Primary key	review_id
Candidate / alternate keys	session_id (one review per session in the single-student core model)
Foreign keys	session_id -> MENTORSHIP_SESSION.session_id
Principal functional dependencies	review_id -> session_id, rating, review_text, submitted_at; session_id -> rating, review_text, submitted_at (one review per session)
Design notes	The reviewer and mentor are derived through Session -> Booking -> Allocation -> Request; they are not duplicated in the review relation. If group sessions are introduced, use a separate session-participant relation and UNIQUE(session_id, reviewer_user_id).

NOTIFICATION_OUTBOX

Durable event record written within the same SQL transaction as a business change.

Specification item	Definition
Attributes	outbox_id BIGINT; aggregate_type VARCHAR(40); aggregate_id BIGINT; event_type VARCHAR(60); payload_json JSON; delivery_status VARCHAR(20); attempt_count INT; created_at DATETIME; next_attempt_at DATETIME NULL; processed_at DATETIME NULL; last_error TEXT NULL; dedupe_key VARCHAR(160)
Primary key	outbox_id
Candidate / alternate keys	dedupe_key
Foreign keys	None (aggregate reference is polymorphic)
Principal functional dependencies	outbox_id -> aggregate_type, aggregate_id, event_type, payload_json, delivery_status, attempt_count, created_at, next_attempt_at, processed_at, last_error, dedupe_key; dedupe_key -> outbox_id, ...

Specification item	Definition
Design notes	One outbox event may fan out to many Notification rows. It has no polymorphic FK. The worker delivers only after commit and uses idempotent recipient delivery.

NOTIFICATION

An inbox item addressed to one user, potentially created from one outbox event.

Specification item	Definition
Attributes	notification_id BIGINT; recipient_user_id BIGINT; outbox_id BIGINT; title VARCHAR(180); body TEXT; created_at DATETIME; read_at DATETIME NULL
Primary key	notification_id
Candidate / alternate keys	(outbox_id, recipient_user_id)
Foreign keys	recipient_user_id -> USER_ACCOUNT.user_id; outbox_id -> NOTIFICATION_OUTBOX.outbox_id
Principal functional dependencies	notification_id -> recipient_user_id, outbox_id, title, body, created_at, read_at; (outbox_id, recipient_user_id) -> notification_id, title, body, created_at, read_at
Design notes	event_type is derived from NOTIFICATION_OUTBOX and is not repeated. One event may create multiple per-recipient inbox rows.

AUDIT_LOG

Append-only record of important actions and before/after values.

Specification item	Definition
Attributes	audit_id BIGINT; actor_user_id BIGINT NULL; entity_type VARCHAR(50); entity_id BIGINT; action VARCHAR(40); old_values_json JSON NULL; new_values_json JSON NULL; occurred_at DATETIME; correlation_id VARCHAR(80) NULL
Primary key	audit_id
Candidate / alternate keys	None assumed
Foreign keys	actor_user_id -> USER_ACCOUNT.user_id (nullable)
Principal functional dependencies	audit_id -> actor_user_id, entity_type, entity_id, action, old_values_json, new_values_json, occurred_at, correlation_id
Design notes	entity_type + entity_id is polymorphic and intentionally is not a conventional FK. Restrict write access and retain logs.

5. Keys, functional dependencies, Armstrong axioms and minimal cover

A functional dependency (FD) is a constraint on every legal state of a relation, not a pattern inferred from a few sample rows. X -> Y means any two rows agreeing on X must also agree on Y. The FDs below follow from candidate keys and explicitly proposed business uniqueness rules. An FD must be justified by the domain and enforced where possible with PRIMARY KEY, UNIQUE, FK or controlled writes.

5.1 Key patterns used in AlumniConnect

Pattern	AlumniConnect example	Reason
Surrogate primary key	user_id, request_id, booking_id	Stable compact identity, independent of mutable descriptive attributes.
Natural alternate key	UserAccount.email; Skill.skill_name; MentorshipDomain.domain_name	Human meaningful value can be unique if normalized and governed.
Composite primary key	StudentSkill(student_user_id, skill_id)	A row represents one student-skill association; neither side alone is unique.
Unique business FK	MentorAllocation.request_id; MentorshipSession.booking_id	Enforces at most one allocation per request and one session per booking.
Partial active uniqueness	Booking.active_availability_id	Enforces one active confirmed booking per slot but allows cancelled booking history.

5.2 Principal dependencies by relation

Relation	Principal functional dependencies
ROLE	role_id -> role_code, role_name, description, is_active; role_code -> role_id, role_name, description, is_active
USER_ACCOUNT	user_id -> all non-key attributes; email -> user_id and all remaining attributes
STUDENT_PROFILE	user_id -> student_number, department_id, programme, batch_year, expected_graduation_year, career_goal, bio; student_number -> user_id and remaining attributes
MENTOR_PROFILE	user_id -> all mentor attributes; non-NULL alumni_identifier -> user_id and remaining attributes
DEPARTMENT	department_id -> department_code, department_name, institution_name; (institution_name, department_code) -> department_id, department_name
COMPANY	company_id -> company_name, website_url, industry
SKILL	skill_id -> skill_name, category, description; skill_name -> skill_id and remaining attributes
STUDENT_SKILL	(student_user_id, skill_id) -> proficiency_level, years_experience, added_at
MENTOR_SKILL	(mentor_user_id, skill_id) -> proficiency_level, years_experience, added_at
MENTORSHIP_DOMAIN	domain_id -> domain_name, description; domain_name -> domain_id, description
STUDENT_DOMAIN_INTEREST	(student_user_id, domain_id) -> priority_order, interest_notes; optionally (student_user_id, priority_order) -> domain_id, interest_notes
MENTOR_DOMAIN_EXPERTISE	(mentor_user_id, domain_id) -> expertise_level, years_experience, is_primary
MENTORSHIP_REQUEST	request_id -> student, mentor, domain, message, status, decision metadata and timestamps; non-NULL idempotency_key -> request_id and remaining attributes; pending_request_key is unique only among PENDING rows

Relation	Principal functional dependencies
RECOMMENDATION_RUN	run_id -> student, goal, algorithm_version, profile_version, filters, executed_at
RECOMMENDED_MENTOR	recommendation_id -> all non-key attributes; (run_id, mentor_user_id) -> rank, score, eligibility, explanation; (run_id, rank_position) -> mentor and candidate details if unique
MENTOR_ALLOCATION	allocation_id -> request_id, status and lifecycle fields; request_id -> allocation_id and lifecycle fields. Student/mentor are derived through request_id.
MENTOR_AVAILABILITY	availability_id -> mentor, start, end, status; (mentor, start, end) -> availability_id and status
BOOKING	booking_id -> allocation, availability, status and timestamps; active_availability_id is unique only among active confirmed rows. Student/mentor/time are derived through referenced relations.
MENTORSHIP_SESSION	session_id -> booking and outcome fields; booking_id -> session_id and outcome fields
SESSION_SKILL_ASSESSMENT	(session_id, skill_id) -> assessment_result, assessment_notes
SESSION_FEEDBACK	feedback_id -> session, submitter and feedback; (session_id, submitted_by_user_id) -> remaining feedback fields
MENTOR_REVIEW	review_id -> session, rating, text and timestamp; session_id -> rating, text and timestamp under one-review-per-session rule; reviewer/mentor are derived through the session chain
NOTIFICATION_OUTBOX	outbox_id -> event fields and delivery state; dedupe_key -> outbox_id and event fields
NOTIFICATION	notification_id -> recipient, outbox link and inbox fields; (outbox_id, recipient_user_id) -> notification details; event_type is derived from the linked outbox row
AUDIT_LOG	audit_id -> actor, target descriptor, action, snapshots and timestamp

FD CAUTION

Do not assert company_name -> company_id, mentor_user_id -> skill_id, student_user_id -> domain_id, or student_user_id -> mentor_user_id. These would incorrectly collapse many-to-many relationships or impose uniqueness the product does not require.

5.3 Types of functional dependency

Type	Meaning	AlumniConnect example
Trivial FD	Y is a subset of X; the dependency adds no new attribute.	(student_user_id, skill_id) -> skill_id.
Non-trivial FD	At least one attribute on the right is not already in X.	(student_user_id, skill_id) -> proficiency_level.
Full functional dependency	No proper subset of X determines Y.	(student_user_id, skill_id) -> proficiency_level, because student_user_id alone and skill_id alone do not determine the relationship-specific level.
Partial dependency	A proper subset of a composite key determines a non-prime attribute.	In a badly combined student-skill relation, student_user_id -> student_name would be partial when the key is (student_user_id, skill_id).
Transitive dependency	A key determines a non-key attribute through another non-key determinant.	student_user_id -> department_id and department_id -> department_name imply student_user_id -> department_name transitively if department_name is copied into STUDENT_PROFILE.

5.4 Armstrong axioms and useful inference rules

Armstrong axioms are sound and complete rules for reasoning about functional dependencies. They help derive F+ (the closure of an FD set), prove whether an attribute set is a key, and remove redundant dependencies when finding a minimal cover.

Rule	Statement	Interpretation
Reflexivity	If Y is a subset of X, then X -> Y.	A set determines any subset of itself.
Augmentation	If X -> Y, then XZ -> YZ.	Adding the same attributes to both sides preserves an FD.
Transitivity	If X -> Y and Y -> Z, then X -> Z.	Dependencies can be chained.
Decomposition	If X -> YZ, then X -> Y and X -> Z.	A multi-attribute right side can be split.
Union	If X -> Y and X -> Z, then X -> YZ.	Dependencies with the same determinant can be combined.
Pseudotransitivity	If X -> Y and WY -> Z, then WX -> Z.	A useful derived rule for longer FD chains.

5.5 Minimal cover (canonical cover)

A minimal cover is an equivalent FD set in which every right-hand side has one attribute, no left-hand-side attribute is extraneous, and no dependency is redundant. A practical procedure is: (1) split multi-attribute right sides; (2) test and remove extraneous determinant attributes using closure; (3) remove each FD in turn if the remaining set still implies it.

```
Example: F = { (student_user_id, skill_id) -> proficiency_level, years_experience, added_at }
Split RHS: { (student_user_id, skill_id) -> proficiency_level,
        (student_user_id, skill_id) -> years_experience,
        (student_user_id, skill_id) -> added_at }
No determinant attribute is extraneous under the stated business rule: neither
student_user_id nor skill_id alone determines any association-specific attribute.
No FD is redundant. This is a minimal cover for the stated relation.
```

IMPORTANT

A UNIQUE constraint is not just a performance index: it asserts a business dependency. Do not add UNIQUE(email), UNIQUE(company_name) or UNIQUE(mentor_id, skill_id) unless the underlying business rule actually requires it.

6. Candidate-key derivation and attribute closure

Attribute closure X+ is the set of all attributes functionally determined by X under the stated FD set. If X+ contains every attribute of a relation, X is a superkey; if no proper subset of X is a superkey, X is a candidate key.

Example 1 - StudentSkill composite key

Relation: STUDENT_SKILL(student_user_id, skill_id, proficiency_level, years_experience, added_at). Proposed FD set:

```
(student_user_id, skill_id) -> proficiency_level, years_experience, added_at
```

Closure: {student_user_id, skill_id}+ = {student_user_id, skill_id, proficiency_level, years_experience, added_at}. Neither {student_user_id}+ nor {skill_id}+ determines the association-specific proficiency. Therefore the pair is a minimal superkey and the candidate key.

Example 2 - UserAccount alternate key

```
user_id -> role_id, email, password_hash, display_name, account_status, created_at, updated_at
email   -> user_id, role_id, password_hash, display_name, account_status, created_at, updated_at
```

If email is declared UNIQUE and NOT NULL, both user_id and email are candidate keys. user_id is selected as the primary key; email is the alternate key. If email can be changed, the surrogate PK remains stable while the unique rule still prevents duplicate active identities.

Example 3 - RecommendedMentor candidate keys

```
recommendation_id -> run_id, mentor_user_id, rank_position, score, eligibility_status, explanation_text
(run_id, mentor_user_id) -> rank_position, score, eligibility_status, explanation_text
(run_id, rank_position) -> mentor_user_id, score, eligibility_status, explanation_text  [if rank is unique]
```

The design chooses recommendation_id as the primary key for convenient references. The two composite unique constraints protect business uniqueness within each recommendation run.

Example 4 - Allocation request uniqueness

```
allocation_id -> request_id, allocation_status, allocated_at, ended_at, termination_reason
request_id -> allocation_id, allocation_status, allocated_at, ended_at, termination_reason  [UNIQUE(request_id)]
```

A request can be accepted only once into a single allocation. The unique request_id constraint is the database-level expression of the 1:0..1 request-to-allocation rule. Participant IDs are intentionally derived via the request rather than stored again.

Example 5 - Booking key and why the availability FK is not a global candidate key

```
booking_id -> allocation_id, availability_id, booking_status, booked_at, cancelled_at, cancellation_reason
idempotency_key -> booking_id and remaining attributes  [UNIQUE, NOT NULL]
active_availability_id -> booking_id  [only for CONFIRMED rows]
```

A cancelled booking remains in history and a slot may later be booked again if policy permits. Therefore availability_id does not uniquely identify every historical BOOKING row. The nullable generated active_availability_id is a partial uniqueness guard for confirmed rows, not a candidate key for the entire relation.

Example 6 - Mentor review alternate candidate key

```
review_id -> session_id, rating, review_text, submitted_at
session_id -> rating, review_text, submitted_at  [UNIQUE(session_id)]
```

The reviewer and reviewed mentor are determined by the linked session's booking/allocation/request chain, so they are not copied into MENTOR_REVIEW. If group sessions are introduced, this one-review-per-session FD no longer holds; introduce session participants and use (session_id, reviewer_user_id).

Example 7 - Partial uniqueness is not a global candidate key

```
pending_request_key is unique only when request_status = 'PENDING'
active_availability_id is unique only when booking_status = 'CONFIRMED'
Both generated values become NULL after the row leaves the active state.
They are partial business guards, not candidate keys for all historical rows.
```

The full relation still uses request_id / booking_id as its primary key. MySQL permits multiple NULL values in a UNIQUE index, which allows resolved history rows to coexist while protecting active work.

How to compute a closure in practice

1. Start with X+ = X.
2. For each FD Y -> Z, if Y is contained in the current closure, add Z.
3. Repeat until no new attributes can be added.
4. Compare the resulting closure with the full relation heading.
5. Test minimality by removing one attribute at a time from X.

> **IMPORTANT**
>
> A list of sample rows cannot prove an FD. The team must establish the business rule (for example, "email uniquely identifies an account") and enforce it with a UNIQUE constraint or other mechanism.

7. Normalization walkthrough: 1NF to BCNF, with 4NF/5NF context

Normalization analyzes relation schemas using functional dependencies and keys to reduce unnecessary redundancy and avoid insertion, update and deletion anomalies. The course material emphasizes 1NF, 2NF, 3NF and BCNF, with lossless join and dependency preservation as additional design requirements.

7.1 Start with an unsafe combined relation

Suppose a first draft stores a student, mentor, skills, domain, request, booking, meeting and review in one wide relation. It may contain repeated values such as “Python, SQL, Docker”, nested booking histories or several feedback entries in one record. This is not a good relational design.

Problem	Anomaly created
Repeated skill lists in one field	Filtering, joining and validating each skill becomes unreliable; violates atomicity.
Mentor name copied into every skill row	Changing a mentor name requires many row updates; update anomaly.
Company name repeated with each mentor	Company data changes can produce inconsistent copies.
Booking columns mixed with session outcome	A planned appointment and the actual meeting become indistinguishable.
Feedback stored as a list or repeated columns	Cannot reliably enforce one feedback per participant or query each response independently.

7.2 First Normal Form (1NF)

1NF requires every attribute value to be atomic for the chosen domain and disallows repeating groups, multi-valued attributes and nested relations. Each row-column intersection holds one value.

```
UNNORMALIZED: StudentProfile(user_id, name, skills = "Python, SQL, Docker")
1NF:          StudentProfile(user_id, name, ...)
              StudentSkill(user_id, skill_id, proficiency_level, ...)
```

- Move each student-skill pair to a separate StudentSkill row. Do the same for mentor skills and domain interests/expertise.
- Move each assessed skill for a session to SessionSkillAssessment rather than storing a comma-separated list.
- Keep one feedback submission per row; do not place a collection of feedback objects in a column.

7.3 Second Normal Form (2NF)

2NF requires 1NF and every non-prime attribute to be fully functionally dependent on every candidate key. A relation with a single-attribute candidate key cannot have a partial dependency on a proper subset of that key.

Illustrative unsafe relation: MENTOR_SKILL_DETAIL(mentor_id, skill_id, mentor_name, company_id, skill_name, proficiency_level). Its candidate key is (mentor_id, skill_id). The following dependencies hold under this example:

```
mentor_id -> mentor_name, company_id
skill_id  -> skill_name
(mentor_id, skill_id) -> proficiency_level
company_id -> company_name  [if company_name had been included]
```

mentor_id -> mentor_name and skill_id -> skill_name are partial dependencies because each depends on only part of the composite key. Decompose into MentorProfile, Company, Skill and MentorSkill. Keep proficiency_level in MentorSkill because it describes the mentor-skill pair.

7.4 Third Normal Form (3NF)

The general 3NF test is: for every non-trivial FD X -> A, either X is a superkey or A is a prime attribute. Informally, non-key attributes should depend on the key, the whole key and nothing but the key.

```
UNSAFE: StudentDetail(user_id, student_number, department_id, department_code, department_name, programme)
FDs:    user_id -> student_number, department_id, programme
        student_number -> user_id, department_id, programme  [if unique]
        department_id -> department_code, department_name
DECOMPOSE:
        StudentProfile(user_id, student_number, department_id, programme, ...)
        Department(department_id, department_code, department_name, ...)
```

department_id -> department_name is a transitive dependency from the student key through department_id. Store department facts once in Department and reference them by a foreign key.

7.5 Boyce-Codd Normal Form (BCNF)

BCNF is stronger than 3NF. For every non-trivial FD X -> A, X must be a superkey. Some relations can satisfy 3NF but fail BCNF when a non-superkey determinant determines a prime attribute. The actual FD set must be checked before claiming BCNF.

> **ALUMNICONNECT TARGET**
>
> The proposed relations are designed to be in 3NF and generally BCNF under the stated business dependencies. This is a design target, not a mathematical proof for unknown future rules. Re-check BCNF whenever a new uniqueness rule or cross-attribute dependency is introduced.

7.6 Lifecycle separation is also a normalization decision

Fact	Owning relation	Why it is not merged
Student intent and mentor decision	MENTORSHIP_REQUEST	The request exists before acceptance and can end as REJECTED or CANCELLED without creating an allocation.
Accepted mentoring relationship	MENTOR_ALLOCATION	The allocation is created only after acceptance and can later end independently.
Planned appointment and reserved slot	BOOKING	A booking reserves one availability interval; it does not prove that a meeting took place.
Actual meeting and outcome	MENTORSHIP_SESSION	The session records completion, no-show/cancellation and outcome details. A cancelled booking may have no session.
Participant feedback and mentor rating	SESSION_FEEDBACK / MENTOR_REVIEW	Feedback may be two-way; the mentor review is a student-authored reputation record after completion.

These are separate business facts with different lifecycles and participation rules. Combining them would create nullable columns and update anomalies, or force a planned appointment to masquerade as a completed session.

7.7 Fourth and Fifth Normal Forms - when they matter

The DBMS Unit 3 material notes that 4NF is based on multivalued dependencies and 5NF on join dependencies. These are not needed for every project relation; apply them only when independent multi-valued facts or non-trivial join dependencies are actually present.

Normal form	Meaning	AlumniConnect application
4NF	A relation should not combine independent non-trivial multivalued dependencies unless their determinant is a superkey.	Do not put a student's skill set and domain-interest set into the same wide row. Keep StudentSkill and StudentDomainInterest separately.
5NF	A relation should not contain a non-trivial join dependency that requires further lossless decomposition.	No specific 5NF violation is established by the blueprint. Do not decompose relations without a real business join dependency.

7.8 Normalization outcome by relation family

Relation family	Key property	Normalization conclusion
Reference tables	One ID or unique natural code determines descriptive columns.	1NF/2NF/3NF; normally BCNF under the stated keys.
Profile tables	A profile key determines only attributes of that profile; department/company facts live separately.	3NF; separate organizational reference entities remove transitive dependencies.
Associative tables	Composite key determines pair-specific attributes.	2NF is checked carefully; pair-specific attributes stay on the bridge.
Request/allocation tables	Surrogate ID and one-to-one unique business FK.	Avoid copying participant fields already determined by request_id.
Booking/session tables	Each lifecycle has its own key and independent state.	Separates planned reservation from actual execution and prevents mixed lifecycle columns.
Feedback/audit/outbox	Each event or submission is a separate row.	History is append-oriented; uniqueness prevents duplicate submissions/events.

8. Lossless join, dependency preservation and integrity

8.1 Lossless (non-additive) join

A decomposition is lossless when joining the decomposed relations recreates exactly the original valid relation without generating spurious tuples. For a binary decomposition R into R1 and R2, the common attributes must functionally determine all attributes in at least one side under F+ (the lossless binary decomposition test taught in Unit 3).

- StudentProfile.department_id references Department.department_id. Joining a student profile to its department by that key returns the department determined by the FK.
- MentorSkill joins to MentorProfile by mentor_user_id and to Skill by skill_id. Joining through the full bridge key reconstructs the mentor-skill association without inventing combinations.
- Do not join a relation using a descriptive attribute such as company_name or skill_name if it is not guaranteed unique; that can create duplicate or spurious combinations.

8.2 Dependency preservation

A decomposition is dependency-preserving when each FD can be checked in an individual resulting relation without requiring an expensive join. This is desirable because unique constraints and checks can be applied locally. The Unit 3 material emphasizes that lossless join is essential and dependency preservation is desirable; when BCNF decomposition loses a dependency, preserve losslessness and document the trade-off.

8.3 Integrity constraints to implement

Constraint class	AlumniConnect rule / example
Entity integrity	Every table has a non-NULL primary key.
Referential integrity	StudentSkill.student_user_id references StudentProfile; mentor skill references MentorProfile; booking references one allocation and one availability slot; session references booking.
Domain integrity	Ratings between 1 and 5; experience >= 0; end time > start time; controlled status values.
Uniqueness	Email, skill name, domain name, student number; one allocation per request; one session per booking.
Business integrity	Only pending request can be accepted/rejected/cancelled; only active allocation can be booked.
Temporal integrity	No overlapping confirmed bookings for a student; one active booking per mentor slot.
Historical integrity	Do not silently delete rejected/cancelled requests or completed sessions; retain audit history.

8.4 Referential action policy

- Use ON DELETE RESTRICT for users and master data referenced by business history. Deactivate accounts, skills or domains instead of hard deleting them.
- Use ON DELETE CASCADE only for purely dependent, disposable bridge rows if hard-delete policy is explicitly approved. It should not erase bookings, sessions, feedback or audit evidence.
- Use ON UPDATE RESTRICT for surrogate identifiers. Identifiers should be immutable; descriptive values can be updated in their owning entity.

SOURCE-OF-TRUTH RULE

MySQL owns the official profile, request, allocation, availability, booking, session, rating and feedback values. Redis cache entries and Neo4j graph projections must be rebuildable from MySQL and must not override transactional decisions.

9. MySQL DDL patterns and representative definitions

The DDL below is a compact implementation baseline for MySQL 8.x / InnoDB. It demonstrates the key and constraint patterns; the full project script should be reviewed against the final database name, exact datatype lengths and deployment version.

MYSQL VERSION NOTE

Use MySQL 8.0.16 or later for enforced CHECK constraints. Generated-column expressions and UNIQUE indexes shown here are intended for MySQL 8.x / InnoDB. Test the complete DDL on the exact server minor version before importing project data.

WHY BOOKING DOES NOT REPEAT PARTICIPANT OR TIME COLUMNS

The current Business Rules Catalogue includes an illustrative booking implementation that copies student_id, mentor_id, start_at and end_at into BOOKING. This specification instead stores allocation_id and availability_id only. The allocation/request chain determines the student and mentor; availability_id determines the mentor and slot interval. That keeps the base BOOKING relation normalized. A view can expose the joined booking context. If the team deliberately stores immutable booking-time snapshots, treat that as a documented denormalization and add consistency enforcement.

9.1 Reference and identity relations

```
CREATE TABLE role (
  role_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  role_code VARCHAR(30) NOT NULL UNIQUE,
  role_name VARCHAR(80) NOT NULL,
  description TEXT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;

CREATE TABLE department (
  department_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  department_code VARCHAR(30) NOT NULL,
  department_name VARCHAR(120) NOT NULL,
  institution_name VARCHAR(160) NOT NULL,
  UNIQUE (institution_name, department_code)
) ENGINE=InnoDB;

CREATE TABLE company (
  company_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  company_name VARCHAR(180) NOT NULL,
  website_url VARCHAR(255),
  industry VARCHAR(100)
) ENGINE=InnoDB;

CREATE TABLE user_account (
  user_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  role_id BIGINT UNSIGNED NOT NULL,
  email VARCHAR(254) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  display_name VARCHAR(120) NOT NULL,
  account_status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  updated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  last_login_at DATETIME(6),
  CONSTRAINT fk_user_role FOREIGN KEY (role_id)
    REFERENCES role(role_id) ON DELETE RESTRICT,
  CONSTRAINT chk_user_status CHECK
    (account_status IN ('ACTIVE','SUSPENDED','DEACTIVATED'))
) ENGINE=InnoDB;
```

9.2 Role-specific profiles

```
CREATE TABLE student_profile (
  user_id BIGINT UNSIGNED PRIMARY KEY,
  student_number VARCHAR(50) NOT NULL UNIQUE,
  department_id BIGINT UNSIGNED NOT NULL,
  programme VARCHAR(120) NOT NULL,
  batch_year SMALLINT NOT NULL,
  expected_graduation_year SMALLINT,
  career_goal TEXT,
  bio TEXT,
  CONSTRAINT fk_student_user FOREIGN KEY (user_id)
    REFERENCES user_account(user_id) ON DELETE RESTRICT,
  CONSTRAINT fk_student_department FOREIGN KEY (department_id)
    REFERENCES department(department_id) ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE mentor_profile (
  user_id BIGINT UNSIGNED PRIMARY KEY,
  alumni_identifier VARCHAR(60) UNIQUE,
  alumni_department_id BIGINT UNSIGNED NOT NULL,
  current_company_id BIGINT UNSIGNED,
  job_title VARCHAR(120) NOT NULL,
  years_experience DECIMAL(4,1) NOT NULL DEFAULT 0,
  max_active_mentees SMALLINT NOT NULL DEFAULT 1,
  accepting_requests BOOLEAN NOT NULL DEFAULT TRUE,
  professional_summary TEXT,
  CONSTRAINT fk_mentor_user FOREIGN KEY (user_id)
    REFERENCES user_account(user_id) ON DELETE RESTRICT,
  CONSTRAINT fk_mentor_department FOREIGN KEY (alumni_department_id)
    REFERENCES department(department_id) ON DELETE RESTRICT,
  CONSTRAINT fk_mentor_company FOREIGN KEY (current_company_id)
    REFERENCES company(company_id) ON DELETE RESTRICT,
  CONSTRAINT chk_mentor_experience CHECK (years_experience >= 0),
  CONSTRAINT chk_mentor_capacity CHECK (max_active_mentees >= 0)
) ENGINE=InnoDB;
```

9.3 Skills and domains

```
CREATE TABLE skill (
  skill_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  skill_name VARCHAR(100) NOT NULL UNIQUE,
  category VARCHAR(80),
  description TEXT
) ENGINE=InnoDB;

CREATE TABLE mentorship_domain (
  domain_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  domain_name VARCHAR(120) NOT NULL UNIQUE,
  description TEXT
) ENGINE=InnoDB;

CREATE TABLE student_skill (
  student_user_id BIGINT UNSIGNED NOT NULL,
  skill_id BIGINT UNSIGNED NOT NULL,
  proficiency_level VARCHAR(20) NOT NULL,
  years_experience DECIMAL(4,1),
  added_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (student_user_id, skill_id),
  FOREIGN KEY (student_user_id) REFERENCES student_profile(user_id),
  FOREIGN KEY (skill_id) REFERENCES skill(skill_id),
  CHECK (years_experience IS NULL OR years_experience >= 0)
) ENGINE=InnoDB;

CREATE TABLE mentor_skill (
  mentor_user_id BIGINT UNSIGNED NOT NULL,
  skill_id BIGINT UNSIGNED NOT NULL,
  proficiency_level VARCHAR(20) NOT NULL,
  years_experience DECIMAL(4,1),
  added_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (mentor_user_id, skill_id),
  FOREIGN KEY (mentor_user_id) REFERENCES mentor_profile(user_id),
  FOREIGN KEY (skill_id) REFERENCES skill(skill_id),
  CHECK (years_experience IS NULL OR years_experience >= 0)
) ENGINE=InnoDB;

CREATE TABLE student_domain_interest (
  student_user_id BIGINT UNSIGNED NOT NULL,
  domain_id BIGINT UNSIGNED NOT NULL,
  priority_order SMALLINT NOT NULL,
  interest_notes TEXT,
  PRIMARY KEY (student_user_id, domain_id),
  UNIQUE (student_user_id, priority_order),
  FOREIGN KEY (student_user_id) REFERENCES student_profile(user_id),
  FOREIGN KEY (domain_id) REFERENCES mentorship_domain(domain_id),
  CHECK (priority_order > 0)
) ENGINE=InnoDB;

CREATE TABLE mentor_domain_expertise (
  mentor_user_id BIGINT UNSIGNED NOT NULL,
  domain_id BIGINT UNSIGNED NOT NULL,
  expertise_level VARCHAR(20) NOT NULL,
  years_experience DECIMAL(4,1),
  is_primary BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (mentor_user_id, domain_id),
  FOREIGN KEY (mentor_user_id) REFERENCES mentor_profile(user_id),
  FOREIGN KEY (domain_id) REFERENCES mentorship_domain(domain_id),
  CHECK (years_experience IS NULL OR years_experience >= 0)
) ENGINE=InnoDB;
```

9.4 Requests, recommendations and allocation

```
CREATE TABLE mentorship_request (
  request_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  student_user_id BIGINT UNSIGNED NOT NULL,
  mentor_user_id BIGINT UNSIGNED NOT NULL,
  domain_id BIGINT UNSIGNED NOT NULL,
  request_message TEXT NOT NULL,
  career_goal_snapshot TEXT,
  request_status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  submitted_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  decision_at DATETIME(6),
  decided_by_user_id BIGINT UNSIGNED,
  decision_reason VARCHAR(120),
  response_note TEXT,
  idempotency_key VARCHAR(100) UNIQUE,
  pending_request_key VARCHAR(100)
    GENERATED ALWAYS AS
      (CASE WHEN request_status = 'PENDING'
            THEN CONCAT(student_user_id, ':', mentor_user_id, ':', domain_id)
            ELSE NULL END) STORED,
  CONSTRAINT fk_request_student FOREIGN KEY (student_user_id)
    REFERENCES student_profile(user_id) ON DELETE RESTRICT,
  CONSTRAINT fk_request_mentor FOREIGN KEY (mentor_user_id)
    REFERENCES mentor_profile(user_id) ON DELETE RESTRICT,
  CONSTRAINT fk_request_domain FOREIGN KEY (domain_id)
    REFERENCES mentorship_domain(domain_id) ON DELETE RESTRICT,
  CONSTRAINT fk_request_decider FOREIGN KEY (decided_by_user_id)
    REFERENCES user_account(user_id) ON DELETE RESTRICT,
  CONSTRAINT chk_request_status
    CHECK (request_status IN ('PENDING','ACCEPTED','REJECTED','CANCELLED')),
  CONSTRAINT chk_request_decision_metadata
    CHECK ((request_status = 'PENDING' AND decision_at IS NULL AND decided_by_user_id IS NULL)
       OR (request_status <> 'PENDING' AND decision_at IS NOT NULL AND decided_by_user_id IS NOT NULL)),
  UNIQUE (pending_request_key),
  INDEX ix_request_student_status (student_user_id, request_status, submitted_at),
  INDEX ix_request_mentor_status (mentor_user_id, request_status, submitted_at)
) ENGINE=InnoDB;
CREATE TABLE recommendation_run (
  run_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  student_user_id BIGINT UNSIGNED NOT NULL,
  input_goal TEXT,
  algorithm_version VARCHAR(40) NOT NULL,
  profile_version VARCHAR(40),
  filter_snapshot JSON,
  executed_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  FOREIGN KEY (student_user_id) REFERENCES student_profile(user_id)
) ENGINE=InnoDB;
CREATE TABLE recommended_mentor (
  recommendation_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  run_id BIGINT UNSIGNED NOT NULL,
  mentor_user_id BIGINT UNSIGNED NOT NULL,
  rank_position SMALLINT NOT NULL,
  score DECIMAL(7,4) NOT NULL,
  eligibility_status VARCHAR(20) NOT NULL,
  explanation_text TEXT,
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  UNIQUE (run_id, mentor_user_id),
  UNIQUE (run_id, rank_position),
  FOREIGN KEY (run_id) REFERENCES recommendation_run(run_id),
  FOREIGN KEY (mentor_user_id) REFERENCES mentor_profile(user_id),
  CHECK (rank_position > 0),
  CHECK (score >= 0)
) ENGINE=InnoDB;
CREATE TABLE mentor_allocation (
  allocation_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  request_id BIGINT UNSIGNED NOT NULL UNIQUE,
  allocation_status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  allocated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  ended_at DATETIME(6),
  ended_by_user_id BIGINT UNSIGNED,
  termination_reason TEXT,
  CONSTRAINT fk_allocation_request FOREIGN KEY (request_id)
    REFERENCES mentorship_request(request_id) ON DELETE RESTRICT,
  CONSTRAINT fk_allocation_ender FOREIGN KEY (ended_by_user_id)
    REFERENCES user_account(user_id) ON DELETE RESTRICT,
  CHECK (allocation_status IN ('ACTIVE','ENDED')),
  CHECK ((allocation_status = 'ACTIVE' AND ended_at IS NULL)
     OR (allocation_status = 'ENDED' AND ended_at IS NOT NULL))
) ENGINE=InnoDB;
```

9.5 Availability, booking and session

```
CREATE TABLE mentor_availability (
  availability_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  mentor_user_id BIGINT UNSIGNED NOT NULL,
  starts_at DATETIME(6) NOT NULL,
  ends_at DATETIME(6) NOT NULL,
  slot_status VARCHAR(20) NOT NULL DEFAULT 'OPEN',
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  UNIQUE (mentor_user_id, starts_at, ends_at),
  CONSTRAINT fk_availability_mentor FOREIGN KEY (mentor_user_id)
    REFERENCES mentor_profile(user_id) ON DELETE RESTRICT,
  CHECK (ends_at > starts_at),
  CHECK (slot_status IN ('OPEN','BOOKED','BLOCKED','EXPIRED')),
  INDEX ix_availability_mentor_time (mentor_user_id, starts_at, slot_status)
) ENGINE=InnoDB;

CREATE TABLE booking (
  booking_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  allocation_id BIGINT UNSIGNED NOT NULL,
  availability_id BIGINT UNSIGNED NOT NULL,
  booking_status VARCHAR(20) NOT NULL DEFAULT 'CONFIRMED',
  booked_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  cancelled_at DATETIME(6),
  cancelled_by_user_id BIGINT UNSIGNED,
  cancellation_reason TEXT,
  idempotency_key VARCHAR(100) NOT NULL UNIQUE,
  active_availability_id BIGINT UNSIGNED
    GENERATED ALWAYS AS
      (CASE WHEN booking_status = 'CONFIRMED'
            THEN availability_id ELSE NULL END) STORED,
  UNIQUE (active_availability_id),
  CONSTRAINT fk_booking_allocation FOREIGN KEY (allocation_id)
    REFERENCES mentor_allocation(allocation_id) ON DELETE RESTRICT,
  CONSTRAINT fk_booking_availability FOREIGN KEY (availability_id)
    REFERENCES mentor_availability(availability_id) ON DELETE RESTRICT,
  CONSTRAINT fk_booking_canceller FOREIGN KEY (cancelled_by_user_id)
    REFERENCES user_account(user_id) ON DELETE RESTRICT,
  CHECK (booking_status IN ('CONFIRMED','CANCELLED')),
  CHECK ((booking_status = 'CONFIRMED' AND cancelled_at IS NULL)
     OR (booking_status = 'CANCELLED' AND cancelled_at IS NOT NULL)),
  INDEX ix_booking_allocation_status (allocation_id, booking_status)
) ENGINE=InnoDB;

CREATE TABLE mentorship_session (
  session_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  booking_id BIGINT UNSIGNED NOT NULL UNIQUE,
  session_status VARCHAR(20) NOT NULL DEFAULT 'SCHEDULED',
  actual_start_at DATETIME(6),
  actual_end_at DATETIME(6),
  discussion_summary TEXT,
  performance_observations TEXT,
  improvement_areas TEXT,
  next_steps TEXT,
  preparation_recommendation TEXT,
  recorded_by_user_id BIGINT UNSIGNED,
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id),
  FOREIGN KEY (recorded_by_user_id) REFERENCES user_account(user_id),
  CHECK (session_status IN ('SCHEDULED','COMPLETED','CANCELLED','NO_SHOW')),
  CHECK (actual_end_at IS NULL OR actual_start_at IS NULL
      OR actual_end_at >= actual_start_at)
) ENGINE=InnoDB;

CREATE TABLE session_skill_assessment (
  session_id BIGINT UNSIGNED NOT NULL,
  skill_id BIGINT UNSIGNED NOT NULL,
  assessment_result VARCHAR(30) NOT NULL,
  assessment_notes TEXT,
  PRIMARY KEY (session_id, skill_id),
  FOREIGN KEY (session_id) REFERENCES mentorship_session(session_id),
  FOREIGN KEY (skill_id) REFERENCES skill(skill_id)
) ENGINE=InnoDB;
```

9.6 Feedback and operations

```
CREATE TABLE session_feedback (
  feedback_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  session_id BIGINT UNSIGNED NOT NULL,
  submitted_by_user_id BIGINT UNSIGNED NOT NULL,
  communication_rating TINYINT,
  helpfulness_rating TINYINT,
  technical_preparation_rating TINYINT,
  comments TEXT,
  submitted_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  UNIQUE (session_id, submitted_by_user_id),
  FOREIGN KEY (session_id) REFERENCES mentorship_session(session_id),
  FOREIGN KEY (submitted_by_user_id) REFERENCES user_account(user_id),
  CHECK (communication_rating IS NULL OR communication_rating BETWEEN 1 AND 5),
  CHECK (helpfulness_rating IS NULL OR helpfulness_rating BETWEEN 1 AND 5),
  CHECK (technical_preparation_rating IS NULL OR technical_preparation_rating BETWEEN 1 AND 5)
) ENGINE=InnoDB;

CREATE TABLE mentor_review (
  review_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  session_id BIGINT UNSIGNED NOT NULL UNIQUE,
  rating TINYINT NOT NULL,
  review_text TEXT,
  submitted_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  CONSTRAINT fk_review_session FOREIGN KEY (session_id)
    REFERENCES mentorship_session(session_id) ON DELETE RESTRICT,
  CHECK (rating BETWEEN 1 AND 5)
) ENGINE=InnoDB;

CREATE TABLE notification_outbox (
  outbox_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  aggregate_type VARCHAR(40) NOT NULL,
  aggregate_id BIGINT UNSIGNED NOT NULL,
  event_type VARCHAR(60) NOT NULL,
  payload_json JSON NOT NULL,
  delivery_status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  attempt_count INT NOT NULL DEFAULT 0,
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  next_attempt_at DATETIME(6),
  processed_at DATETIME(6),
  last_error TEXT,
  dedupe_key VARCHAR(160) NOT NULL UNIQUE,
  CHECK (delivery_status IN ('PENDING','PROCESSING','SENT','FAILED'))
) ENGINE=InnoDB;

CREATE TABLE notification (
  notification_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  recipient_user_id BIGINT UNSIGNED NOT NULL,
  outbox_id BIGINT UNSIGNED NOT NULL,
  title VARCHAR(180) NOT NULL,
  body TEXT NOT NULL,
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  read_at DATETIME(6),
  UNIQUE (outbox_id, recipient_user_id),
  FOREIGN KEY (recipient_user_id) REFERENCES user_account(user_id) ON DELETE RESTRICT,
  FOREIGN KEY (outbox_id) REFERENCES notification_outbox(outbox_id) ON DELETE RESTRICT,
  INDEX ix_notification_inbox (recipient_user_id, read_at, created_at)
) ENGINE=InnoDB;

CREATE TABLE audit_log (
  audit_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  actor_user_id BIGINT UNSIGNED,
  entity_type VARCHAR(50) NOT NULL,
  entity_id BIGINT UNSIGNED NOT NULL,
  action VARCHAR(40) NOT NULL,
  old_values_json JSON,
  new_values_json JSON,
  occurred_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  correlation_id VARCHAR(80),
  FOREIGN KEY (actor_user_id) REFERENCES user_account(user_id)
) ENGINE=InnoDB;
```

10. Triggers, stored procedures, transaction boundaries and outbox

The DBMS Unit 2 material defines triggers as automatic actions caused by INSERT, UPDATE or DELETE events and stored procedures as named server-side programs with IN / OUT / INOUT parameters. In AlumniConnect, use triggers for last-line row-level validation and optional audit capture; use procedures for actor-aware, multi-row workflows, capacity checks, locking, booking overlap checks and outbox insertion.

10.1 What belongs in a trigger vs a procedure

Use a trigger for	Use a stored procedure for
Validate a row-level state transition before the change.	Accept/reject a request and create its allocation atomically.
Capture OLD and NEW values for important changes when direct DML is permitted.	Book a slot after locking student, mentor, allocation and availability rows.
Reject invalid row-local status transitions or values.	Insert outbox events and coordinate all related records in the same transaction.
Enforce simple row-local rules that apply to all writers.	Check cross-row rules, capacity, overlap and coordinated multi-table updates.

DO NOT SEND EMAIL FROM A TRIGGER

A trigger must not call external APIs, send email or depend on a network service. The business procedure writes one transactional outbox event before COMMIT; a worker delivers it only after commit and retries failures safely.

10.2 Trigger - validate request state transitions

Allowed transition from the blueprint: PENDING -> ACCEPTED, REJECTED or CANCELLED. Once a request has left PENDING, the status cannot be changed again through this trigger. If the team later needs reopen/resubmit, define a separate explicit transition and audit it.

```
DELIMITER $$
CREATE TRIGGER trg_request_status_guard
BEFORE UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
 IF OLD.request_status <> NEW.request_status THEN
  IF OLD.request_status <> 'PENDING'
    OR NEW.request_status NOT IN ('ACCEPTED','REJECTED','CANCELLED') THEN
   SIGNAL SQLSTATE '45000'
     SET MESSAGE_TEXT = 'Invalid mentorship request status transition';
  END IF;
 END IF;
END$$
DELIMITER ;
```

10.3 Trigger - audit request status changes (optional)

This optional AFTER trigger captures OLD and NEW state values for every request status update. A database trigger cannot automatically know the authenticated web user unless a trusted application context is propagated to the same connection. The example therefore writes a NULL actor for system/database-level evidence. In a deployment that requires human attribution, write the actor-specific audit row inside the stored procedure and disable this trigger to avoid duplicate audit rows.

```
DELIMITER $$
CREATE TRIGGER trg_request_status_audit
AFTER UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
 IF OLD.request_status <> NEW.request_status THEN
   INSERT INTO audit_log
     (actor_user_id, entity_type, entity_id, action,
      old_values_json, new_values_json, occurred_at)
   VALUES
     (NULL, 'MENTORSHIP_REQUEST', NEW.request_id, 'STATUS_CHANGED',
      JSON_OBJECT('status', OLD.request_status),
      JSON_OBJECT('status', NEW.request_status), UTC_TIMESTAMP(6));
 END IF;
END$$
DELIMITER ;
```

Outbox insertion is deliberately handled by the business procedure rather than this trigger. The procedure knows the actor, event payload and recipient set, and can use a stable dedupe_key. Both the audit row and outbox row participate in the surrounding transaction and roll back with it.

10.4 Trigger - validate session lifecycle

```
DELIMITER $$
CREATE TRIGGER trg_session_status_guard
BEFORE UPDATE ON mentorship_session
FOR EACH ROW
BEGIN
 IF OLD.session_status <> NEW.session_status THEN
   IF OLD.session_status <> 'SCHEDULED'
     OR NEW.session_status NOT IN ('COMPLETED','CANCELLED','NO_SHOW') THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid session status transition';
   END IF;
 END IF;

 IF NEW.actual_start_at IS NOT NULL
    AND NEW.actual_end_at IS NOT NULL
    AND NEW.actual_end_at < NEW.actual_start_at THEN
   SIGNAL SQLSTATE '45000'
     SET MESSAGE_TEXT = 'Session end cannot precede session start';
 END IF;

 IF NEW.session_status = 'COMPLETED'
    AND (NEW.actual_start_at IS NULL OR NEW.actual_end_at IS NULL) THEN
   SIGNAL SQLSTATE '45000'
     SET MESSAGE_TEXT = 'Completed session requires actual start and end times';
 END IF;
END$$
DELIMITER ;
```

10.5 Trigger - feedback eligibility

Session feedback is allowed only after the session has been completed and only for one of the actual participants. The database trigger is a last-line integrity guard; the stored procedure and FastAPI authorization layer must still validate the authenticated actor and keep duplicate submissions idempotent.

```
DELIMITER $$
CREATE TRIGGER trg_feedback_participant_guard
BEFORE INSERT ON session_feedback
FOR EACH ROW
BEGIN
 DECLARE v_student BIGINT UNSIGNED DEFAULT NULL;
 DECLARE v_mentor BIGINT UNSIGNED DEFAULT NULL;
 DECLARE v_status VARCHAR(20) DEFAULT NULL;

 SELECT mr.student_user_id, mr.mentor_user_id, ms.session_status
   INTO v_student, v_mentor, v_status
 FROM mentorship_session ms
 JOIN booking b ON b.booking_id = ms.booking_id
 JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
 JOIN mentorship_request mr ON mr.request_id = ma.request_id
 WHERE ms.session_id = NEW.session_id;

 IF v_status IS NULL OR v_status <> 'COMPLETED' THEN
   SIGNAL SQLSTATE '45000'
     SET MESSAGE_TEXT = 'Feedback requires a completed session';
 END IF;

 IF NEW.submitted_by_user_id NOT IN (v_student, v_mentor) THEN
   SIGNAL SQLSTATE '45000'
     SET MESSAGE_TEXT = 'Only a session participant may submit feedback';
 END IF;
END$$
DELIMITER ;
```

The UNIQUE(session_id, submitted_by_user_id) constraint is still required. A trigger check alone cannot prevent concurrent duplicate submissions.

10.6 Trigger - protect allocation and booking status transitions

These row-level guards prevent direct SQL updates from reopening ended allocations or confirming a cancelled booking. The application should route business changes through stored procedures, while triggers provide a final defense for writers that reach the tables directly.

```
DELIMITER $$
CREATE TRIGGER trg_allocation_status_guard
BEFORE UPDATE ON mentor_allocation
FOR EACH ROW
BEGIN
 IF OLD.allocation_status <> NEW.allocation_status THEN
  IF OLD.allocation_status <> 'ACTIVE' OR NEW.allocation_status <> 'ENDED' THEN
   SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Allocation can transition only from ACTIVE to ENDED';
  END IF;
 END IF;
END$$

CREATE TRIGGER trg_booking_status_guard
BEFORE UPDATE ON booking
FOR EACH ROW
BEGIN
 IF OLD.booking_status <> NEW.booking_status THEN
  IF OLD.booking_status <> 'CONFIRMED' OR NEW.booking_status <> 'CANCELLED' THEN
   SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Booking can transition only from CONFIRMED to CANCELLED';
  END IF;
 END IF;
 IF NEW.booking_status = 'CANCELLED' AND NEW.cancelled_at IS NULL THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Cancelled booking requires cancelled_at';
 END IF;
END$$
DELIMITER ;
```

10.7 Trigger - protect mentor review eligibility

The review row intentionally stores only session_id, rating and review text. The student reviewer and mentor being reviewed are derived through the single-student session chain. The trigger confirms that the session is completed; the procedure additionally checks that the authenticated actor is the student participant.

```
DELIMITER $$
CREATE TRIGGER trg_mentor_review_completed_guard
BEFORE INSERT ON mentor_review
FOR EACH ROW
BEGIN
 IF NOT EXISTS (
  SELECT 1
  FROM mentorship_session ms
  JOIN booking b ON b.booking_id = ms.booking_id
  JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
  JOIN mentorship_request mr ON mr.request_id = ma.request_id
  WHERE ms.session_id = NEW.session_id
   AND ms.session_status = 'COMPLETED'
 ) THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Mentor review requires a completed session';
 END IF;
END$$
DELIMITER ;
```

10.8 Stored procedure - accept a mentorship request

The procedure owns the transaction. To prevent two concurrent acceptances from exceeding mentor capacity, every code path that creates or ends an allocation must lock the relevant mentor_profile row. Lock ordering is standardized as Student Profile -> Mentor Profile -> Request. The procedure reads the participant IDs first without locking, locks the profiles in order, then locks and re-reads the request before writing.

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_accept_mentorship_request(
 IN p_request_id BIGINT UNSIGNED,
 IN p_actor_user_id BIGINT UNSIGNED,
 OUT p_allocation_id BIGINT UNSIGNED
)
BEGIN
 DECLARE v_student BIGINT UNSIGNED;
 DECLARE v_mentor BIGINT UNSIGNED;
 DECLARE v_request_status VARCHAR(20);
 DECLARE v_capacity INT;
 DECLARE v_active INT DEFAULT 0;
 DECLARE v_student_account_status VARCHAR(20);
 DECLARE v_mentor_account_status VARCHAR(20);
 DECLARE v_accepting BOOLEAN;
 DECLARE EXIT HANDLER FOR SQLEXCEPTION
 BEGIN
  ROLLBACK;
  RESIGNAL;
 END;

 SET p_allocation_id = NULL;

 -- Discover lock targets before transaction start; the request is re-read FOR UPDATE.
 SELECT student_user_id, mentor_user_id
  INTO v_student, v_mentor
 FROM mentorship_request
 WHERE request_id = p_request_id;
 IF v_student IS NULL OR v_mentor IS NULL THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Mentorship request does not exist';
 END IF;

 START TRANSACTION;

 -- Global lock order: student, mentor, request.
 SELECT ua.account_status INTO v_student_account_status
 FROM student_profile sp
 JOIN user_account ua ON ua.user_id = sp.user_id
 WHERE sp.user_id = v_student
 FOR UPDATE;

 SELECT ua.account_status, mp.max_active_mentees, mp.accepting_requests
  INTO v_mentor_account_status, v_capacity, v_accepting
 FROM mentor_profile mp
 JOIN user_account ua ON ua.user_id = mp.user_id
 WHERE mp.user_id = v_mentor
 FOR UPDATE;

 SELECT student_user_id, mentor_user_id, request_status
  INTO v_student, v_mentor, v_request_status
 FROM mentorship_request
 WHERE request_id = p_request_id
 FOR UPDATE;

 IF v_request_status <> 'PENDING' THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Request is not pending';
 END IF;
 IF v_mentor <> p_actor_user_id THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Only the addressed mentor may accept';
 END IF;
 IF v_student_account_status <> 'ACTIVE'
   OR v_mentor_account_status <> 'ACTIVE'
   OR v_accepting <> TRUE THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Student or mentor account is not eligible';
 END IF;

 SELECT COUNT(*) INTO v_active
 FROM mentor_allocation ma
 JOIN mentorship_request mr ON mr.request_id = ma.request_id
 WHERE mr.mentor_user_id = v_mentor
  AND ma.allocation_status = 'ACTIVE';

 IF v_active >= v_capacity THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Mentor capacity has been reached';
 END IF;
```

```
  INSERT INTO mentor_allocation(request_id, allocation_status)
  VALUES (p_request_id, 'ACTIVE');
  SET p_allocation_id = LAST_INSERT_ID();

  UPDATE mentorship_request
  SET request_status = 'ACCEPTED',
     decision_at = UTC_TIMESTAMP(6),
     decided_by_user_id = p_actor_user_id
  WHERE request_id = p_request_id;

  INSERT INTO audit_log
   (actor_user_id, entity_type, entity_id, action,
    old_values_json, new_values_json, occurred_at)
  VALUES
   (p_actor_user_id, 'MENTORSHIP_REQUEST', p_request_id, 'REQUEST_ACCEPTED',
    JSON_OBJECT('status', 'PENDING'),
    JSON_OBJECT('status', 'ACCEPTED', 'allocation_id', p_allocation_id),
    UTC_TIMESTAMP(6));

  INSERT INTO notification_outbox
   (aggregate_type, aggregate_id, event_type, payload_json,
    delivery_status, attempt_count, created_at, dedupe_key)
  VALUES
   ('MENTORSHIP_REQUEST', p_request_id, 'REQUEST_ACCEPTED',
    JSON_OBJECT('request_id', p_request_id,
              'student_user_id', v_student,
              'mentor_user_id', v_mentor,
              'allocation_id', p_allocation_id,
              'recipient_user_ids', JSON_ARRAY(v_student)),
    'PENDING', 0, UTC_TIMESTAMP(6),
    CONCAT('request:', p_request_id, ':accepted'));

  COMMIT;
 END$$
 DELIMITER ;
```

The UNIQUE(mentor_allocation.request_id) key is a second line of defence. The procedure must be called as a standalone transaction; do not wrap it in a second application-managed transaction. If the request was accepted by a concurrent call, the second call must return an already-processed conflict and must not create a second allocation.

10.9 Stored procedure - create a confirmed booking atomically

The procedure validates the active allocation, confirms that the slot belongs to the allocation mentor, locks both participants and the availability row, checks overlap for both student and mentor, reserves the slot, creates the session and records the outbox event in one transaction. The generated unique active_availability_id is the final guard against double booking of one slot.

```
DELIMITER $$
CREATE PROCEDURE sp_create_confirmed_booking(
 IN p_allocation_id BIGINT UNSIGNED,
 IN p_availability_id BIGINT UNSIGNED,
 IN p_actor_user_id BIGINT UNSIGNED,
 IN p_idempotency_key VARCHAR(100),
 OUT p_booking_id BIGINT UNSIGNED
)
booking_proc: BEGIN
 DECLARE v_student BIGINT UNSIGNED;
 DECLARE v_mentor BIGINT UNSIGNED;
 DECLARE v_allocation_status VARCHAR(20);
 DECLARE v_slot_mentor BIGINT UNSIGNED;
 DECLARE v_start DATETIME(6);
 DECLARE v_end DATETIME(6);
 DECLARE v_slot_status VARCHAR(20);
 DECLARE v_overlap INT DEFAULT 0;
 DECLARE v_student_status VARCHAR(20);
 DECLARE v_mentor_status VARCHAR(20);
 DECLARE EXIT HANDLER FOR SQLEXCEPTION
 BEGIN
  ROLLBACK;
  RESIGNAL;
 END;

 SET p_booking_id = NULL;

 -- Discover the lock targets before opening the transaction. This avoids
 -- establishing a REPEATABLE READ snapshot before participant locks are acquired.
 SELECT mr.student_user_id, mr.mentor_user_id
  INTO v_student, v_mentor
 FROM mentor_allocation ma
 JOIN mentorship_request mr ON mr.request_id = ma.request_id
 WHERE ma.allocation_id = p_allocation_id;
 IF v_student IS NULL OR v_mentor IS NULL THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Allocation does not exist';
 END IF;

 IF p_idempotency_key IS NULL THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'An idempotency key is required';
 END IF;

 -- Idempotency lookup is outside the transaction so it cannot establish a stale
 -- REPEATABLE READ snapshot before the participant locks. It returns a replay only
 -- for the same actor, allocation and slot. A conflicting reuse is rejected by UNIQUE.
 SELECT MAX(b.booking_id) INTO p_booking_id
 FROM booking b
 JOIN mentor_allocation ma2 ON ma2.allocation_id = b.allocation_id
 JOIN mentorship_request mr2 ON mr2.request_id = ma2.request_id
 WHERE b.idempotency_key = p_idempotency_key
  AND mr2.student_user_id = p_actor_user_id
  AND b.allocation_id = p_allocation_id
  AND b.availability_id = p_availability_id;
 IF p_booking_id IS NOT NULL THEN
  LEAVE booking_proc;
 END IF;

 IF v_student <> p_actor_user_id THEN
  SIGNAL SQLSTATE '45000'
   SET MESSAGE_TEXT = 'Only the allocation student may create the booking';
 END IF;

 START TRANSACTION;

 -- Global lock order serializes overlap checks for both participants.
 SELECT ua.account_status INTO v_student_status
 FROM student_profile sp
 JOIN user_account ua ON ua.user_id = sp.user_id
 WHERE sp.user_id = v_student
 FOR UPDATE;

 SELECT ua.account_status INTO v_mentor_status
 FROM mentor_profile mp
 JOIN user_account ua ON ua.user_id = mp.user_id
 WHERE mp.user_id = v_mentor
```

```
  FOR UPDATE;

  SELECT ma.allocation_status, mr.student_user_id, mr.mentor_user_id
    INTO v_allocation_status, v_student, v_mentor
  FROM mentor_allocation ma
  JOIN mentorship_request mr ON mr.request_id = ma.request_id
  WHERE ma.allocation_id = p_allocation_id
  FOR UPDATE;

  IF v_allocation_status <> 'ACTIVE'
     OR v_student_status <> 'ACTIVE'
     OR v_mentor_status <> 'ACTIVE' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'An active allocation and active participants are required';
  END IF;

  -- Lock the availability row before validating slot state and time.
  SELECT mentor_user_id, starts_at, ends_at, slot_status
    INTO v_slot_mentor, v_start, v_end, v_slot_status
  FROM mentor_availability
  WHERE availability_id = p_availability_id
  FOR UPDATE;

  IF v_slot_mentor IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Availability slot does not exist';
  END IF;
  IF v_slot_status <> 'OPEN' OR v_start <= UTC_TIMESTAMP(6) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Availability slot is not open or is in the past';
  END IF;
  IF v_slot_mentor <> v_mentor THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Selected slot does not belong to the allocation mentor';
  END IF;

  -- Half-open interval overlap: existing.start < requested.end AND existing.end > requested.start.
  SELECT COUNT(*) INTO v_overlap
  FROM booking b
  JOIN mentor_availability a ON a.availability_id = b.availability_id
  JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
  JOIN mentorship_request mr ON mr.request_id = ma.request_id
  WHERE b.booking_status = 'CONFIRMED'
    AND (mr.student_user_id = v_student OR mr.mentor_user_id = v_mentor)
    AND a.starts_at < v_end
    AND a.ends_at > v_start;

  IF v_overlap > 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Student or mentor has an overlapping confirmed booking';
  END IF;

  INSERT INTO booking
    (allocation_id, availability_id, booking_status, idempotency_key)
  VALUES
    (p_allocation_id, p_availability_id, 'CONFIRMED', p_idempotency_key);
  SET p_booking_id = LAST_INSERT_ID();

  UPDATE mentor_availability
  SET slot_status = 'BOOKED'
  WHERE availability_id = p_availability_id;

  INSERT INTO mentorship_session(booking_id, session_status)
  VALUES (p_booking_id, 'SCHEDULED');

  INSERT INTO notification_outbox
    (aggregate_type, aggregate_id, event_type, payload_json,
     delivery_status, attempt_count, created_at, dedupe_key)
  VALUES
    ('BOOKING', p_booking_id, 'BOOKING_CONFIRMED',
     JSON_OBJECT('booking_id', p_booking_id,
                 'recipient_user_ids', JSON_ARRAY(v_student, v_mentor)),
     'PENDING', 0, UTC_TIMESTAMP(6),
     CONCAT('booking:', p_booking_id, ':confirmed'));

  COMMIT;
END$$
DELIMITER ;
```

10.10 Stored procedure - record a session outcome

Only the assigned mentor may record the offline session outcome. The application must authenticate the actor; the procedure checks the mentor through Session -> Booking -> Allocation -> Request and permits only SCHEDULED -> COMPLETED / CANCELLED / NO_SHOW.

```
DELIMITER $$
CREATE PROCEDURE sp_record_session_outcome(
  IN p_session_id BIGINT UNSIGNED,
  IN p_actor_user_id BIGINT UNSIGNED,
  IN p_final_status VARCHAR(20),
  IN p_actual_start_at DATETIME(6),
  IN p_actual_end_at DATETIME(6),
  IN p_discussion_summary TEXT,
  IN p_observations TEXT,
  IN p_improvement_areas TEXT,
  IN p_next_steps TEXT,
  IN p_preparation_recommendation TEXT
)
BEGIN
  DECLARE v_mentor BIGINT UNSIGNED;
  DECLARE v_student BIGINT UNSIGNED;
  DECLARE v_status VARCHAR(20);
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;
  START TRANSACTION;
  SELECT mr.student_user_id, mr.mentor_user_id, ms.session_status
    INTO v_student, v_mentor, v_status
  FROM mentorship_session ms
  JOIN booking b ON b.booking_id = ms.booking_id
  JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
  JOIN mentorship_request mr ON mr.request_id = ma.request_id
  WHERE ms.session_id = p_session_id
  FOR UPDATE;
  IF v_mentor <> p_actor_user_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only the assigned mentor may record the outcome';
  END IF;
  IF v_status <> 'SCHEDULED'
     OR p_final_status NOT IN ('COMPLETED','CANCELLED','NO_SHOW') THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Invalid session outcome';
  END IF;
  IF p_final_status = 'COMPLETED'
     AND (p_actual_start_at IS NULL OR p_actual_end_at IS NULL
          OR p_actual_end_at < p_actual_start_at) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Completed session requires valid actual times';
  END IF;
  UPDATE mentorship_session
  SET session_status = p_final_status,
      actual_start_at = p_actual_start_at,
      actual_end_at = p_actual_end_at,
      discussion_summary = p_discussion_summary,
      performance_observations = p_observations,
      improvement_areas = p_improvement_areas,
      next_steps = p_next_steps,
      preparation_recommendation = p_preparation_recommendation,
      recorded_by_user_id = p_actor_user_id
  WHERE session_id = p_session_id;
  INSERT INTO audit_log
    (actor_user_id, entity_type, entity_id, action,
     old_values_json, new_values_json, occurred_at)
  VALUES
    (p_actor_user_id, 'MENTORSHIP_SESSION', p_session_id, 'SESSION_OUTCOME_RECORDED',
     JSON_OBJECT('status', 'SCHEDULED'),
     JSON_OBJECT('status', p_final_status), UTC_TIMESTAMP(6));


  INSERT INTO notification_outbox
    (aggregate_type, aggregate_id, event_type, payload_json,
     delivery_status, attempt_count, created_at, dedupe_key)
  VALUES
    ('MENTORSHIP_SESSION', p_session_id, 'SESSION_OUTCOME_RECORDED',
     JSON_OBJECT('session_id', p_session_id,
                 'recipient_user_ids', JSON_ARRAY(v_student, v_mentor)),
     'PENDING', 0, UTC_TIMESTAMP(6),
     CONCAT('session:', p_session_id, ':', p_final_status));


  COMMIT;
END$$
DELIMITER ;
```

10.11 Stored procedure - submit a mentor review

```
DELIMITER $$
CREATE PROCEDURE sp_submit_mentor_review(
  IN p_session_id BIGINT UNSIGNED,
  IN p_actor_user_id BIGINT UNSIGNED,
  IN p_rating TINYINT,
  IN p_review_text TEXT
)
BEGIN
  DECLARE v_student BIGINT UNSIGNED;
  DECLARE v_session_status VARCHAR(20);
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
  SELECT mr.student_user_id, ms.session_status
    INTO v_student, v_session_status
  FROM mentorship_session ms
  JOIN booking b ON b.booking_id = ms.booking_id
  JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
  JOIN mentorship_request mr ON mr.request_id = ma.request_id
  WHERE ms.session_id = p_session_id
  FOR UPDATE;

  IF v_student <> p_actor_user_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Only the student participant may review the mentor';
  END IF;
  IF v_session_status <> 'COMPLETED' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Mentor review requires a completed session';
  END IF;
  IF p_rating < 1 OR p_rating > 5 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Rating must be between 1 and 5';
  END IF;

  INSERT INTO mentor_review(session_id, rating, review_text)
  VALUES (p_session_id, p_rating, p_review_text);

  INSERT INTO notification_outbox
    (aggregate_type, aggregate_id, event_type, payload_json,
     delivery_status, attempt_count, created_at, dedupe_key)
  VALUES
    ('MENTOR_REVIEW', p_session_id, 'MENTOR_REVIEW_SUBMITTED',
     JSON_OBJECT('session_id', p_session_id,
                 'student_user_id', p_actor_user_id),
     'PENDING', 0, UTC_TIMESTAMP(6),
     CONCAT('review:', p_session_id, ':submitted'));

  COMMIT;
END$$
DELIMITER ;
```

10.12 Other useful procedures to add

Procedure	Core responsibility
sp_submit_mentorship_request	Verify active student/mentor accounts and required domain; reject duplicate pending request; insert PENDING request plus REQUEST_SUBMITTED outbox event.
sp_reject_mentorship_request	Lock request; verify target mentor and PENDING state; record decision time/reason; notify student.
sp_cancel_mentorship_request	Allow only requesting student to cancel a PENDING request; retain history.
sp_publish_mentor_slot	Verify mentor ownership, future start/end, no overlapping OPEN slots; insert OPEN availability.
sp_cancel_booking	Authorize participant; lock booking/slot; set CANCELLED; mark any SCHEDULED session CANCELLED; reopen slot only if policy permits; preserve rows and queue both notifications in one transaction.

Procedure	Core responsibility
sp_reschedule_booking	Preserve original booking; cancel/supersede it, validate new slot with same overlap rules, create replacement and link it to original.
sp_submit_session_feedback	Require completed session; verify student or mentor participant; enforce UNIQUE(session_id, submitted_by_user_id).
sp_end_allocation	Lock mentor and allocation; set ACTIVE -> ENDED; record timestamp, actor and reason; prevent future booking.
sp_process_notification_outbox	Claim pending events in batches; deliver after commit; retry with backoff; dedupe recipients; mark SENT or FAILED.

10.13 Transaction, ACID, isolation, locking and cross-store consistency

ACID property	AlumniConnect application
Atomicity	Accept-request transaction commits allocation + request status + audit + outbox together. Booking transaction commits booking + availability state + scheduled session + outbox together.
Consistency	PK/FK/UNIQUE/CHECK constraints, state guards and procedures preserve legal request, allocation, booking, session and review states.
Isolation	Concurrent acceptance and booking operations lock shared parent profile rows before checking capacity or overlapping appointments. The chosen lock order must be consistent.
Durability	After COMMIT, InnoDB redo/undo and recovery protect committed rows according to the database durability configuration.

Lock order and concurrency pattern

Use a predictable lock order for every path that touches the same business objects: StudentProfile -> MentorProfile -> Request/Allocation -> Availability -> Booking/Session. A transaction that ends an allocation or cancels a booking must follow a compatible order. Inconsistent lock order increases deadlock risk.

- Mentor capacity: serialize acceptance operations by locking the MentorProfile row before reading active allocation count and inserting a new allocation. The initial request read is only to discover lock targets; re-read the request under FOR UPDATE after participant locks.
- Student/mentor overlap: serialize bookings by locking both participant profile rows, then the allocation and availability slot, then check for overlapping CONFIRMED bookings.
- Use half-open time intervals [start, end). Two intervals overlap when existing_start < requested_end AND existing_end > requested_start. Adjacent intervals (10:00-10:30 and 10:30-11:00) do not overlap.
- A UNIQUE generated active_availability_id prevents two CONFIRMED bookings against the same slot even when concurrent code paths race. A profile lock is still required for overlap checks across different slots.
- Retry deadlock victims and lock-wait timeouts at the application boundary using the same idempotency key. Retry only transactions designed to be safe to execute again.

Isolation-level guidance

Isolation level	Use / implication
READ UNCOMMITTED	Not recommended for booking or allocation decisions; dirty reads are unacceptable.
READ COMMITTED	A practical option for workflow transactions because each ordinary read obtains a fresh committed snapshot. Parent-row locks still serialize the relevant participants.
REPEATABLE READ	MySQL InnoDB default. Be careful: a plain consistent read can use a snapshot created before a lock wait. Discover lock targets before START TRANSACTION, then lock and re-read authoritative rows; use current/locking reads for critical predicates or set READ COMMITTED for the workflow transaction.

Isolation level	Use / implication
SERIALIZABLE	Usually unnecessary for this project and may reduce concurrency. Use explicit row locks and unique constraints for the known invariants.

Transactional outbox and polyglot persistence

The SQL transaction writes NotificationOutbox beside the business update. After COMMIT, a worker claims events with a short locking transaction (MySQL 8 can use FOR UPDATE SKIP LOCKED for queue workers), marks them PROCESSING, delivers notifications or projects updates, and marks SENT. Failures increment attempt_count and set next_attempt_at / last_error. Dedupe keys make repeated worker runs idempotent.

- Redis invalidation and Neo4j projection happen after MySQL commit. They are eventually consistent and rebuildable; they are never used to approve an allocation or confirm a slot.
- Do not perform HTTP calls, send email or update Neo4j from a MySQL trigger. Triggers run inside the SQL transaction and external side effects cannot be rolled back.
- Availability.slot_status = BOOKED is a transaction-maintained operational state that mirrors the active booking relation. The booking procedure and cancellation procedure must update both in one transaction; include a reconciliation query in operations tests.

Example reconciliation queries

```
-- Slot marked BOOKED without a confirmed booking (should return no rows)
SELECT a.availability_id
FROM mentor_availability a
LEFT JOIN booking b
  ON b.availability_id = a.availability_id
 AND b.booking_status = 'CONFIRMED'
WHERE a.slot_status = 'BOOKED'
  AND b.booking_id IS NULL;

-- Confirmed booking whose slot is not marked BOOKED (should return no rows)
SELECT b.booking_id, a.availability_id, a.slot_status
FROM booking b
JOIN mentor_availability a ON a.availability_id = b.availability_id
WHERE b.booking_status = 'CONFIRMED'
  AND a.slot_status <> 'BOOKED';
```

TRANSACTION RULE

Never split a booking reservation into “check availability” in one transaction followed by “insert booking” in another. The check, slot update, booking row, session row and outbox event form one atomic transaction.

11. Indexing, views, CTEs and query optimization

11.1 Index candidates

Relation	Recommended index	Workload supported
USER_ACCOUNT	UNIQUE(email); INDEX(role_id, account_status)	Login and role-based administration.
MENTOR_PROFILE	INDEX(accepting_requests, years_experience)	Mentor eligibility and filter lists.
STUDENT_SKILL	PK(student_user_id, skill_id); INDEX(skill_id, student_user_id)	Student skill lookup and reverse mentor matching.
MENTOR_SKILL	PK(mentor_user_id, skill_id); INDEX(skill_id, mentor_user_id)	Find mentors with target skills.
MENTORSHIP_REQUEST	UNIQUE(pending_request_key); INDEX(student_user_id, request_status, submitted_at); INDEX(mentor_user_id, request_status, submitted_at)	Prevent duplicate pending requests; support student history and mentor inbox.
RECOMMENDED_MENTOR	UNIQUE(run_id, mentor_user_id); UNIQUE(run_id, rank_position)	Preserve one candidate per run and one candidate per rank.
MENTOR_ALLOCATION	UNIQUE(request_id); INDEX(allocation_status)	Enforce one allocation per request and list active allocations.
MENTOR_AVAILABILITY	UNIQUE(mentor_user_id, starts_at, ends_at); INDEX(mentor_user_id, starts_at)	Mentor calendar and slot discovery.
BOOKING	UNIQUE(active_availability_id); INDEX(allocation_id, booking_status)	Prevent duplicate active slot booking and list allocation bookings.
MENTORSHIP_SESSION	UNIQUE(booking_id); INDEX(session_status, actual_start_at)	Session retrieval and outcome reporting.
SESSION_FEEDBACK	UNIQUE(session_id, submitted_by_user_id); INDEX(session_id)	Duplicate prevention and session feedback summaries.
NOTIFICATION	INDEX(recipient_user_id, read_at, created_at)	User inbox and unread notifications.
NOTIFICATION_OUTBOX	INDEX(delivery_status, next_attempt_at, created_at)	Worker queue polling.

11.2 Views for safe, reusable reporting

A view provides a reusable logical projection. It is useful for hiding private columns and for exposing normalized relationships without copying them into base tables.

- Create a public mentor directory view that exposes display_name, job_title, company, skills, domains and public summary, but omits email, password hash and private contact data.
- Create a student mentorship-history view joining requests, allocations, bookings and sessions. Do not materialize a second copy of status data unless a measured reporting requirement justifies it.
- Create mentor-rating aggregates from completed-session reviews. Calculate average rating from MentorReview rows rather than storing a writable duplicate average in MentorProfile.

```
CREATE VIEW vw_booking_details AS
SELECT b.booking_id, b.booking_status,
    mr.student_user_id, mr.mentor_user_id,
    a.starts_at, a.ends_at,
    ma.allocation_status
FROM booking b
JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
JOIN mentorship_request mr ON mr.request_id = ma.request_id
JOIN mentor_availability a ON a.availability_id = b.availability_id;
```

11.3 User-defined functions - when they are useful

A MySQL function returns one value and is appropriate for deterministic calculations reused in SELECT expressions. Do not use a function to create bookings, send notifications or perform multi-row workflow changes; those belong in stored procedures.

```
DELIMITER $$
CREATE FUNCTION fn_rating_band(p_rating DECIMAL(3,2))
RETURNS VARCHAR(12)
DETERMINISTIC
BEGIN
  RETURN CASE
    WHEN p_rating IS NULL THEN 'UNRATED'
    WHEN p_rating >= 4.50 THEN 'EXCELLENT'
    WHEN p_rating >= 3.50 THEN 'GOOD'
    WHEN p_rating >= 2.50 THEN 'AVERAGE'
    ELSE 'LOW'
  END;
END$$
DELIMITER ;

-- Example use with a completed-review aggregate query:
SELECT mr2.mentor_user_id,
       AVG(r.rating) AS average_rating,
       fn_rating_band(AVG(r.rating)) AS rating_band
FROM mentor_review r
JOIN mentorship_session ms ON ms.session_id = r.session_id
JOIN booking b ON b.booking_id = ms.booking_id
JOIN mentor_allocation ma ON ma.allocation_id = b.allocation_id
JOIN mentorship_request mr2 ON mr2.request_id = ma.request_id
WHERE ms.session_status = 'COMPLETED'
GROUP BY mr2.mentor_user_id;
```

11.3 Query patterns worth demonstrating for DBMS assessment

DBMS concept	AlumniConnect query example
Joins	List mentors with their skills, employer and expertise domains.
Nested / correlated subqueries	Find students without any active allocation; mentors above average experience in a domain.
Aggregates, GROUP BY, HAVING	Calculate average rating and completed-session count; filter mentors with enough completed sessions.
CTEs	Track funnel: requests submitted -> accepted -> allocated -> booked -> completed.
Window functions	Rank mentors within a domain by score, rating or completed-session outcomes.
EXPLAIN / EXPLAIN ANALYZE	Compare mentor discovery and student history query plans before and after indexes.
Views	Expose a privacy-safe mentor directory and consistent analytics definitions.
User-defined functions	Create a deterministic helper such as a normalized score-band or profile-completeness value only when the calculation is reusable and side-effect free.

INDEXING RULE

Create indexes for demonstrated query patterns and foreign-key joins, not for every column. Measure with realistic generated data and execution plans; an index adds write and storage cost.

12. Test matrix, implementation checklist and open decisions

12.1 Integrity and normalization test matrix

Test ID	Scenario	Expected result
RD-01	Insert two accounts with same email.	Second insert rejected by UNIQUE(email).
RD-02	Insert duplicate Skill.skill_name.	Second insert rejected; skill labels remain canonical.
RD-03	Insert same student-skill pair twice.	Second row rejected by composite primary key.
RD-04	Insert same mentor-domain pair twice.	Second row rejected by composite primary key.
RD-05	Accept one request concurrently from two sessions.	Only one allocation can be created; status guard and UNIQUE(request_id) reject the second.
RD-06	Book the same slot concurrently for two allocations.	Only one active booking succeeds; unique active slot index rejects the other.
RD-07	Same student attempts overlapping bookings with two mentors.	Student row lock serializes the check; second overlapping booking is rejected.
RD-08	Create two sessions for one booking.	Second insert rejected by UNIQUE(booking_id).
RD-09	Submit feedback as a non-participant.	Trigger/procedure rejects submission.
RD-10	Submit feedback twice as same participant for same session.	Second insert rejected by UNIQUE(session_id, submitted_by_user_id).
RD-11	Submit mentor review before session completion.	Rejected by validation procedure/trigger.
RD-12	Submit second mentor review for same session.	Rejected by UNIQUE(session_id).
RD-13	Change a completed session back to SCHEDULED.	Rejected by status transition trigger.
RD-14	Rollback request acceptance after allocation insert.	Allocation, request status and outbox event all roll back together.
RD-15	Force outbox delivery failure.	Event remains retryable with attempts / next_attempt_at; no business transaction is rolled back.
RD-16	Join decomposed student, department and skill relations.	Rows are reconstructed without spurious combinations when joined on declared keys.
RD-17	Submit a second PENDING request for the same student/mentor/domain.	Generated pending_request_key UNIQUE guard rejects it; a resolved request releases the partial key.
RD-18	Cancel a confirmed booking.	Booking history remains; slot reopening follows the approved policy and updates slot state in the same transaction.
RD-19	Retry booking request with the same idempotency key.	Original booking is returned; no second booking/session/outbox event is created.
RD-20	Attempt a booking after allocation becomes ENDED.	Procedure rejects the booking even if Redis or Neo4j still shows old eligibility.
RD-21	Join an inbox notification to its outbox event.	The event type is obtained from NOTIFICATION_OUTBOX; NOTIFICATION does not store a duplicate event_type.
RD-22	Set slot_status to BOOKED without a confirmed booking.	Reconciliation query identifies the inconsistent slot; procedures must keep states atomic.

12.2 Implementation sequence

6. Freeze role policy, status vocabularies, request targeting and cancellation/rebooking policy.
7. Create reference relations: Role, Department, Company, Skill and MentorshipDomain.

8. Create UserAccount and role-specific profiles; verify natural-key uniqueness assumptions.
9. Create bridge relations for skills and domains; test composite keys and normalization.
10. Create request, recommendation, allocation, availability, booking and session relations.
11. Add feedback, reviews, outbox, notifications and audit history.
12. Add CHECK, UNIQUE and FK constraints before writing triggers and procedures.
13. Implement transactional stored procedures and row-level triggers; test rollback and concurrency.
14. Create workload-driven indexes; capture EXPLAIN / EXPLAIN ANALYZE evidence.
15. Only after MySQL is authoritative, project required relationships to Neo4j and cache rebuildable data in Redis.

12.3 Open decisions requiring team approval

Decision	Why it matters
Can one user hold multiple roles?	Determines UserAccount.role_id vs a UserRole associative relation.
Can a student have several active allocations?	Determines whether active student-allocation uniqueness is required.
Can an active mentor accept new students during temporary unavailability?	Separates capacity from availability slot publication.
Can cancelled availability slots be booked again?	Determines whether a cancelled booking frees the slot or requires a new availability record.
Can both mentor and student submit session feedback?	Confirms participant cardinality and required rating dimensions.
Does every completed session require a mentor review?	Determines whether review participation is optional or mandatory at session close.
Are mentor review edits permitted?	Determines correction/history policy; do not overwrite history silently.
Is department_code globally unique?	If not, use UNIQUE(institution_id, department_code) or another natural candidate key.
Can mentor/company data be private or hidden?	Affects public views and role-based authorization.

Appendix A. Glossary and design references

Term	Meaning in this specification
1NF	All relation attributes have atomic values; no repeating groups or nested sets.
2NF	1NF plus every non-prime attribute fully depends on every candidate key.
3NF	For every non-trivial FD X -> A, X is a superkey or A is prime.
BCNF	For every non-trivial FD X -> A, X is a superkey.
Candidate key	Minimal superkey. One candidate key is selected as the primary key.
Functional dependency	A constraint stating that an attribute set determines another attribute set.
Attribute closure	Set of attributes functionally implied by a starting attribute set.
Lossless join	Decomposition can be rejoined without losing information or creating spurious tuples.
Dependency preservation	Dependencies can be enforced in individual decomposed relations without joins.
Associative relation	A relation representing an M:N relationship; it often has a composite primary key.
Transactional outbox	An event row written in the same database transaction as the business change; external delivery occurs asynchronously after commit.
Idempotency key	Client/request token used to prevent retries from creating duplicate business operations.

Design references used

- Alumni–Student Mentorship Platform: DBMS Project Blueprint - project vision, roles, database responsibilities, domain entities, relationship cardinalities, source-of-truth rule, request/allocation/booking/session/feedback workflows and scope priorities (especially pp. 1-13 and 20-34).
- AlumniConnect Product Requirements Document - actors, functional scope, system boundary, booking-integrity and traceability success indicators (pp. 1-13).
- AlumniConnect Conceptual ER Model - 24 conceptual entities, 36 relationships, cardinalities and business rules (especially pp. 2-15).
- AlumniConnect Business Rules Catalogue - identity, request, allocation, availability, booking, session, feedback/review, outbox, audit, triggers, procedures and concurrency rules (pp. 3-31).
- AlumniConnect Application Architecture - modular monolith and database ownership boundaries (pp. 1-20).
- PES University DBMS Unit 3 material - superkeys/candidate keys, prime/non-prime attributes, functional dependencies, closures and minimal covers, 1NF, 2NF, 3NF, BCNF, lossless join and dependency preservation.
- PES University DBMS Unit 2 material - MySQL trigger definition and syntax, OLD/NEW row references, SIGNAL SQLSTATE error handling, stored procedure parameters and CALL syntax.

This specification deliberately retains the blueprint's distinction between authoritative MySQL data, Redis performance/temporary data and Neo4j relationship projections. The logical schema is intended to be reviewed and agreed by the team before physical deployment.